



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn

南京大学计算机学院

2025-12-23



第十章 文件、外部排序 与外部搜索



- 主存储器和外存储器
- 文件组织
- 多级索引结构
- 外排序



10.4.5 B 树的删除



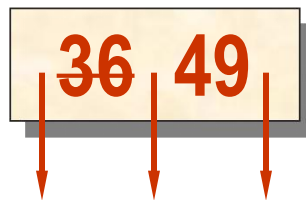
- 在B 树上删除一个关键码时，若结点中所剩关键码个数少于下限，要考虑结点的调整或合并问题，删除过程如下：
 - ✓ 首先需要找到这个关键码所在的结点，从中删去这个关键码。
 - ✓ 若该结点不是叶结点，且被删关键码为 K_i , $1 \leq i \leq n$, 则在删去该关键码之后, 应以该结点 P_i 所指示子树中的最小关键码 x 来代替被删关键码 K_i 所在的位置;
 - ✓ 然后在 x 所在的叶结点中删除 x 。



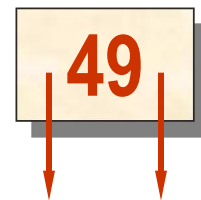
在叶结点上的删除有 4 种情况。

- ① 被删关键码所在叶结点同时是根结点且删除前该结点中关键码个数 $n \geq 2$ ，则直接删去该关键码，并将修改后的结点写回磁盘。

$m = 3$



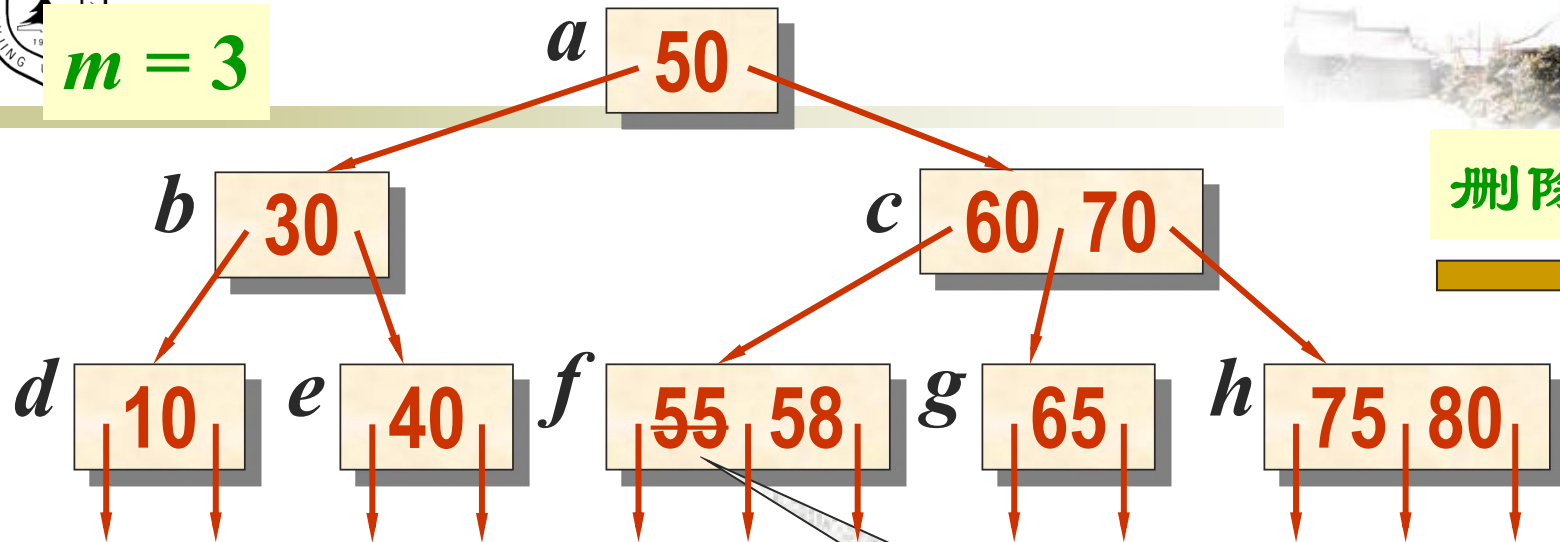
删除36



- ② 被删关键码所在叶结点不是根结点且删除前该结点中关键码个数 $n \geq \lceil m/2 \rceil$ ，则直接删去该关键码并将修改后的结点写回磁盘，删除结束。



$m = 3$

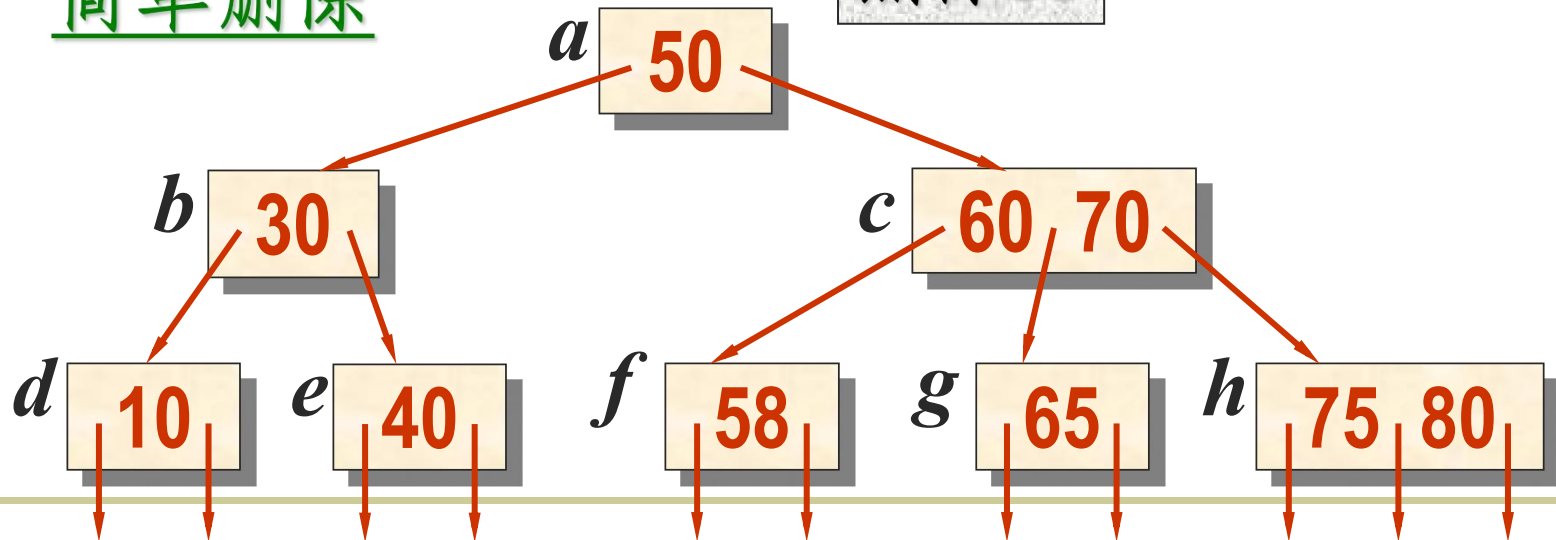


删除55



简单删除

删除55





- ③ 被删关键码所在叶结点删除前关键码个数 $n = \lceil m/2 \rceil - 1$, 若这时与该结点相邻的右兄弟 (或左兄弟) 结点的键码个数 $n \geq \lceil m/2 \rceil$, 则可按以下步骤调整该结点、右兄弟 (或左兄弟) 结点以及其双亲, 以达到新的平衡。
- a) 将双亲结点中刚刚大于 (或小于) 该被删键码的键码 K_i ($1 \leq i \leq n$) 下移;
 - b) 将右兄弟 (或左兄弟) 结点中的最小 (或最大) 键码上移到双亲结点的 K_i 位置;



- c) 将右兄弟 (或左兄弟) 结点中的最左 (或最右) 子树指针平移到被删关键码所在结点中最后 (或最前) 子树指针位置;
- d) 在右兄弟 (或左兄弟) 结点中, 将被移走的关键码和指针位置用剩余的关键码和指针填补、调整。再将结点中的关键码个数减1。



$m = 3$

a 50

b 30

c 60 70

d 10

e 40

f 55 58

g 65

h 75 80

删除65



调整g,c,h

结点联合调整

a 50

b 30

c 60 75

d 10

e 40

f 55 58

g 70

h 80

删除65



④ 被删关键码所在叶结点删除前关键码个数 $n = \lceil m/2 \rceil - 1$, 若这时与该结点相邻的右兄弟 (或左兄弟) 结点的键码个数 $n = \lceil m/2 \rceil - 1$, 则必须按以下步骤合并这两个结点。



- a) 若要合并 p 中的子树指针 P_i 与 P_{i+1} 所指的结点, 且保留 P_i 所指结点, 则把 p 中的关键码 K_{i+1} 下移到 P_i 所指的结点中。
- b) 把 p 中子树指针 P_{i+1} 所指结点中的全部指针和关键码都照搬到 P_i 所指结点的后面。删去 P_{i+1} 所指的结点。
- c) 在结点 p 中用后面剩余的关键码和指针填补关键码 K_{i+1} 和指针 P_{i+1} 。
- d) 修改结点 p 和选定保留结点的关键码个数。



- 在合并结点的过程中, 双亲结点中的关键码个数减少了。
- 若双亲结点是根结点且结点关键码个数减到0, 则将该双亲结点删去, 合并后保留的结点成为新的根结点; 否则将双亲结点与合并后保留的结点都写回磁盘, 删除处理结束。
- 若双亲结点不是根结点且关键码个数减到 $\lceil m/2 \rceil - 2$, 又要与它自己的兄弟结点合并, 重复上面的合并步骤。最坏情况下, 这种结点合并处理要自下向上直到根结点。



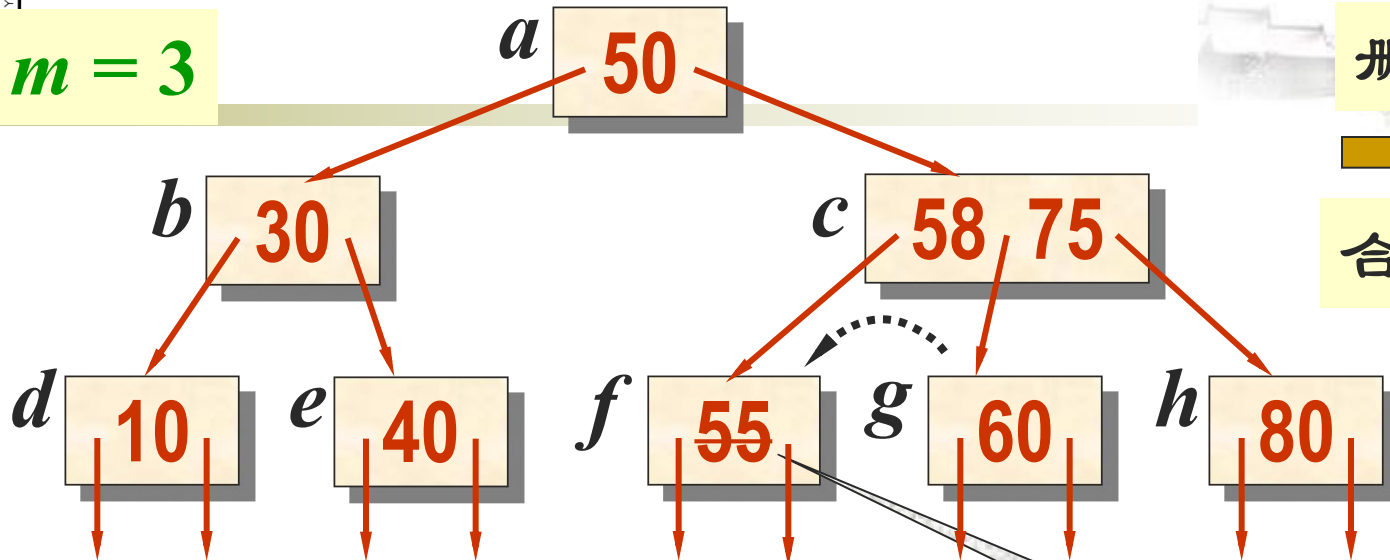
$m = 3$



删除55

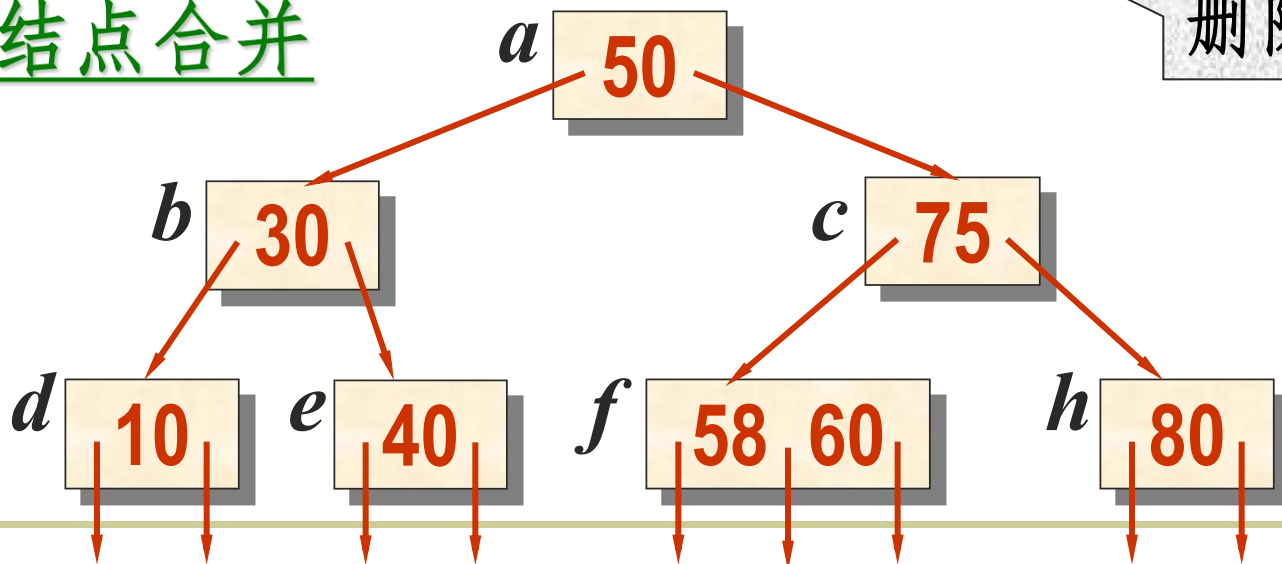


合并f, g



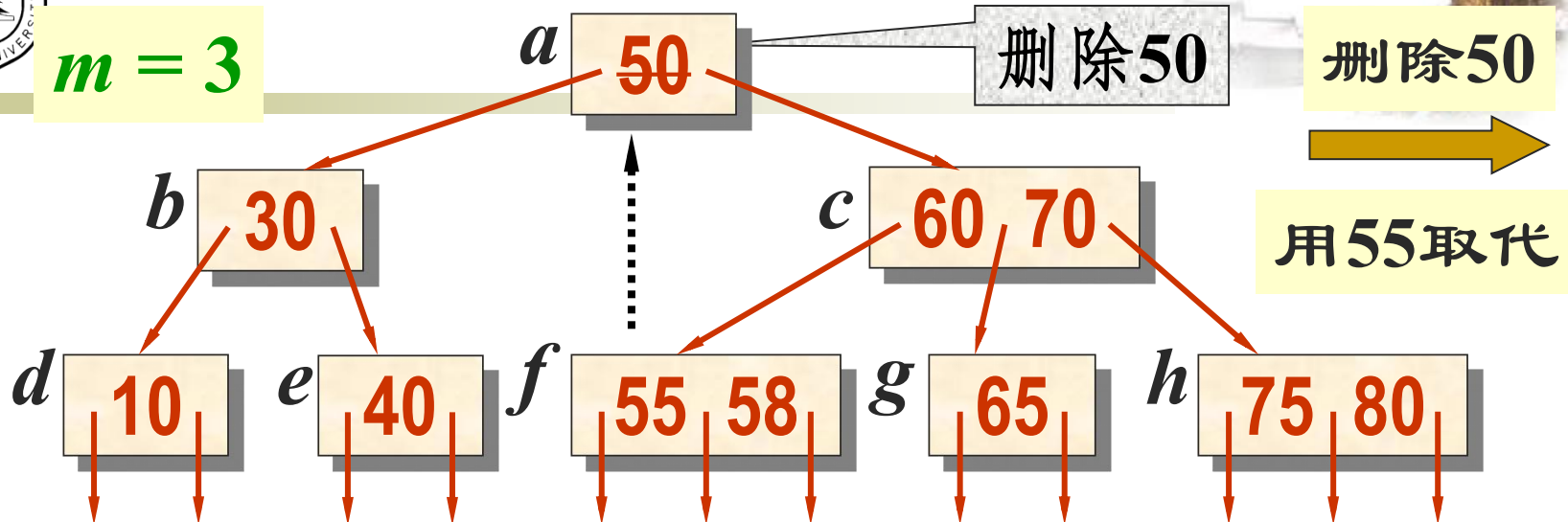
结点合并

删除55

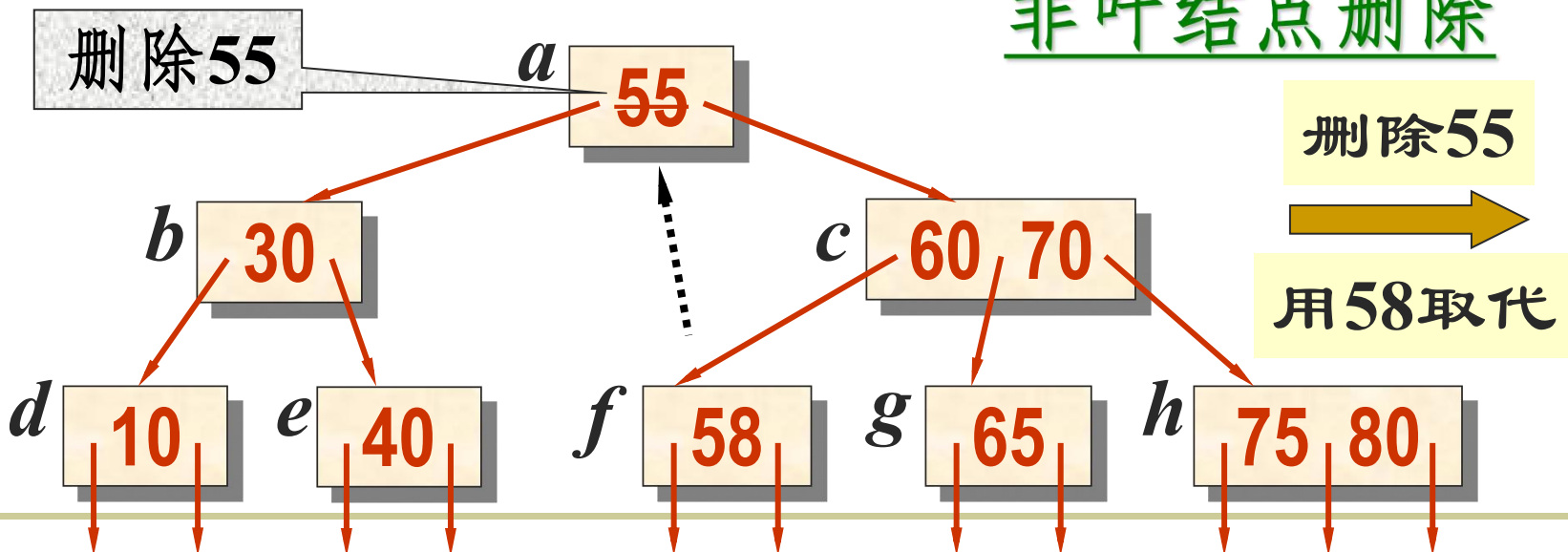


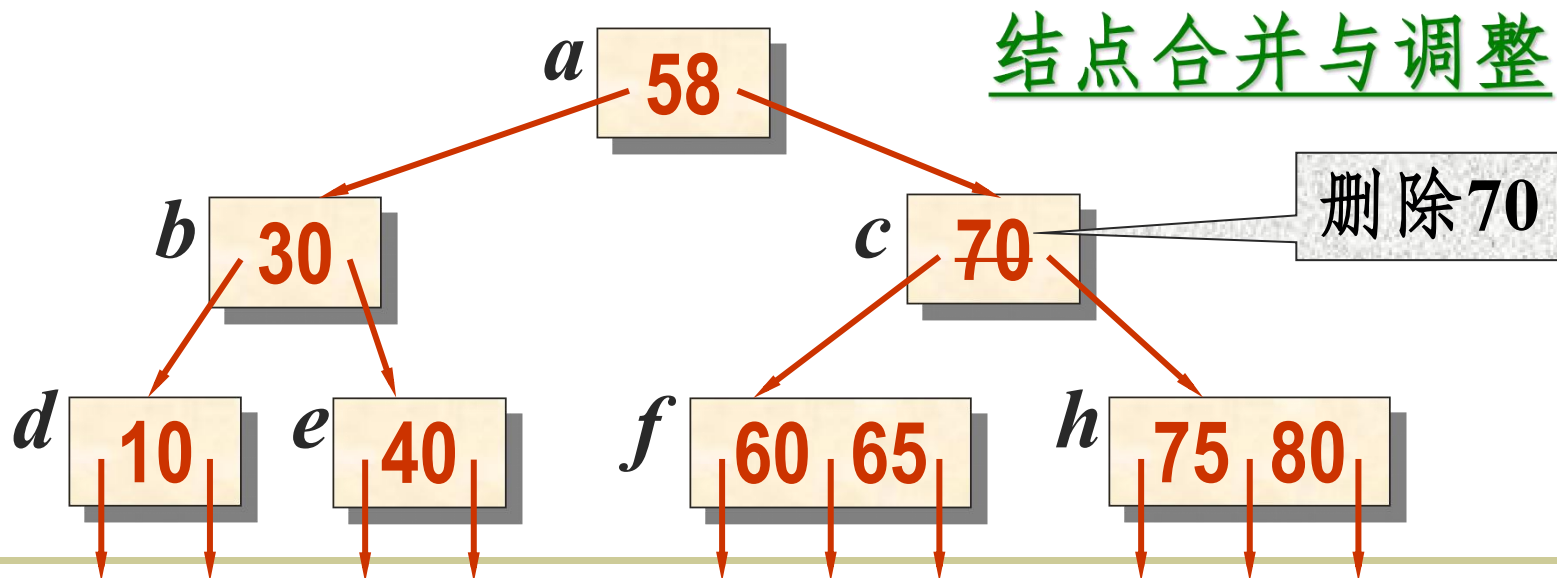
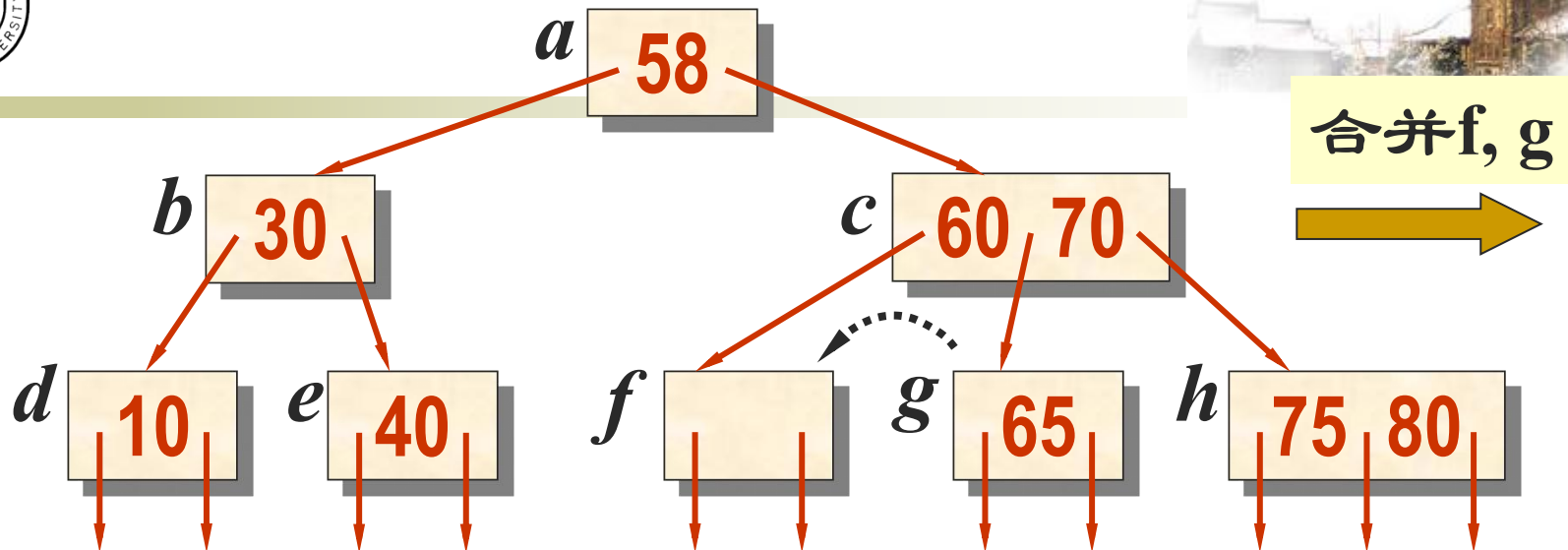


$m = 3$



非叶结点删除



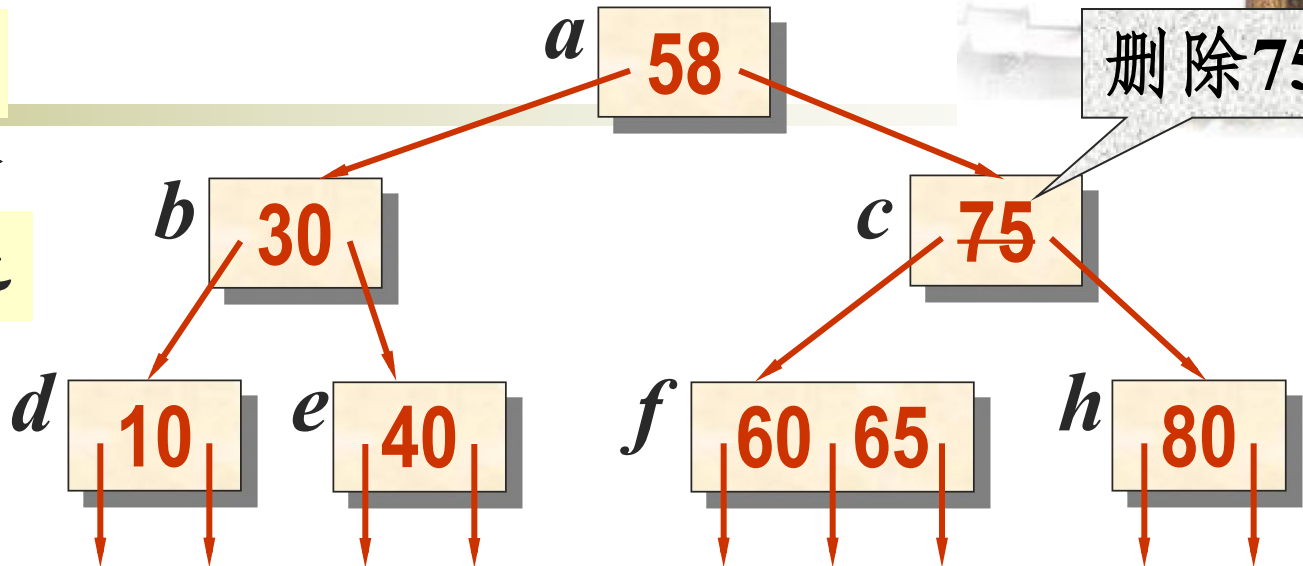




删除70



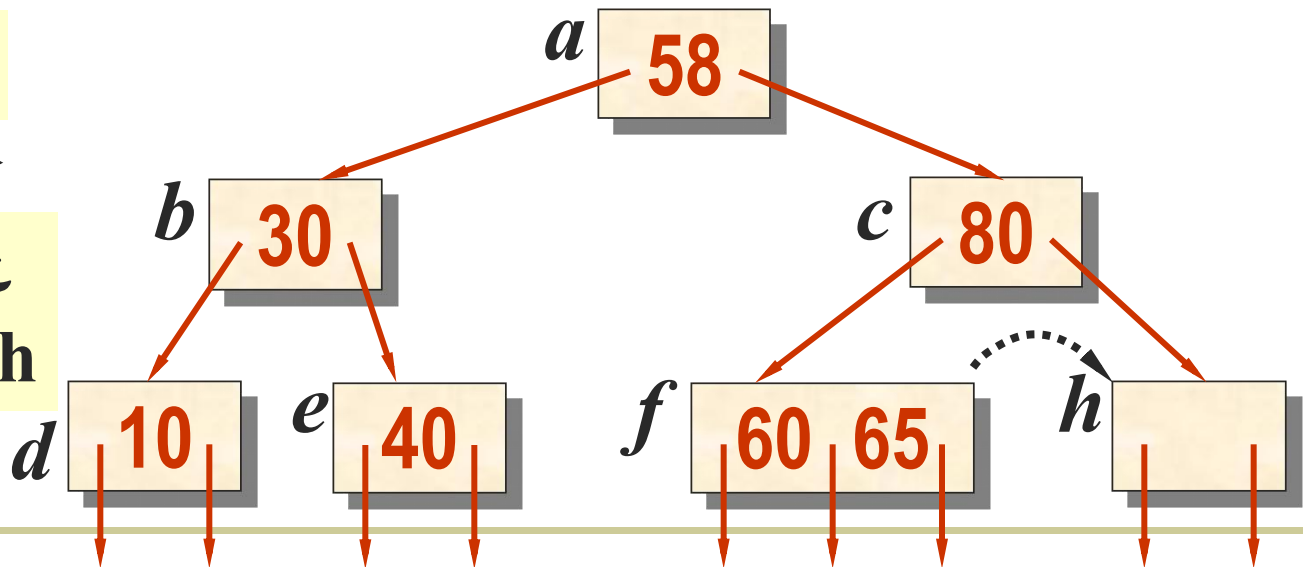
用75取代



删除75

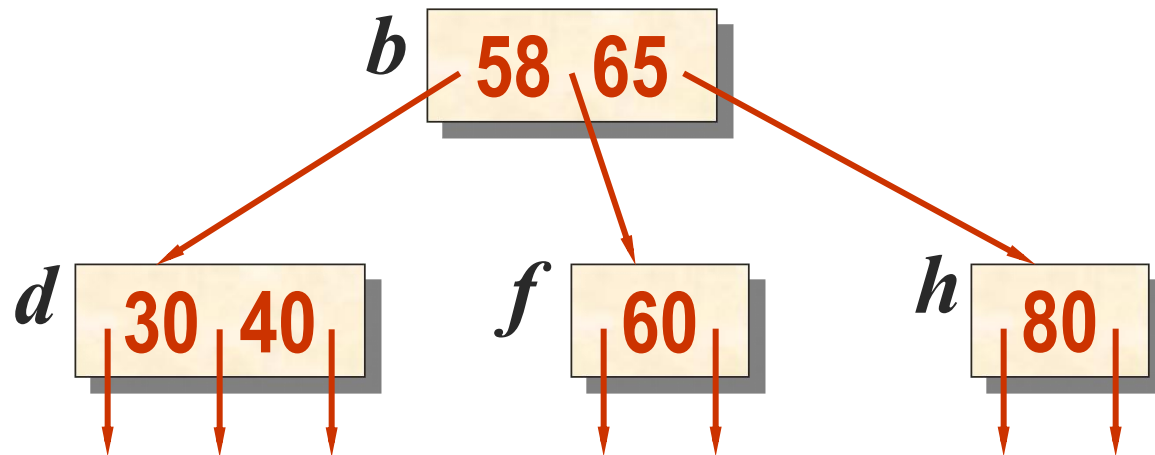
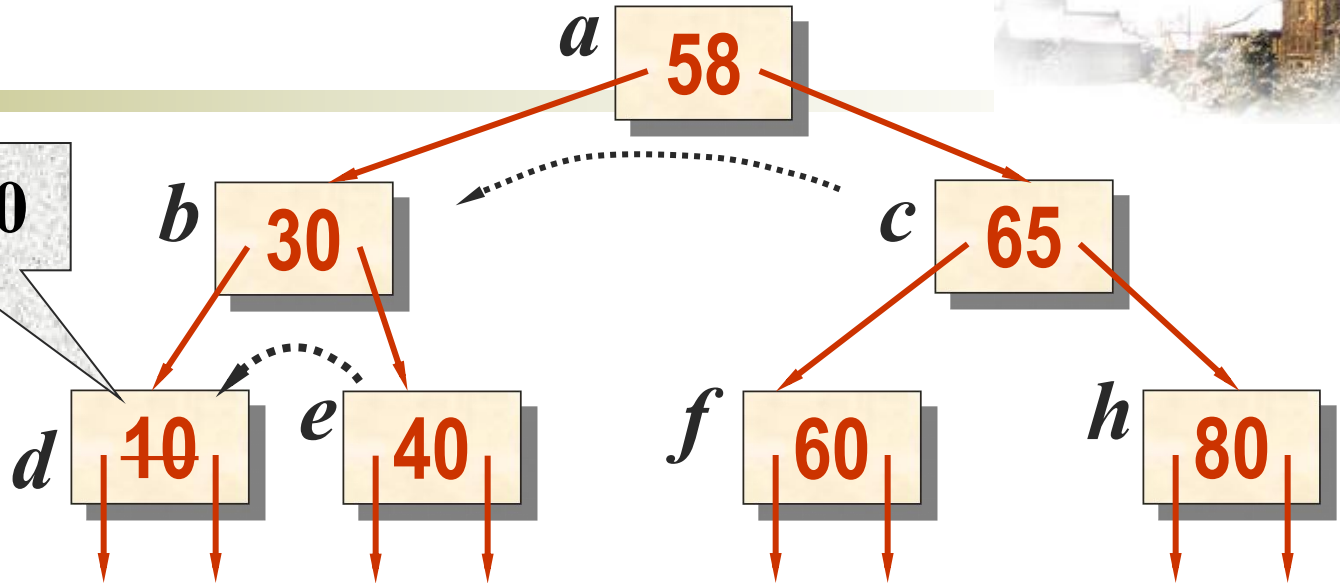


用80取代
调整f, c, h





删除10





B 树

插入

通过“查找”确定插入位置（一定是在终端结点）

若插入后结点关键字个数未超过上限，则无需做其他处理

若插入后关键字个数超过上限，则需要将当前结点的中间元素放到父节点中，当前结点分裂为两个部分；该操作会导致父节点关键字个数+1，若父节点关键字个数也超过了上限，则需要再向上分裂；根节点的分裂会导致B树高度+1。

删除

非终端结点关键字

用其直接前驱或直接后继替代其位置，转化为对“终端结点”的删除

直接前驱：当前关键字左边指针所指子树中“最右下”的元素

直接后继：当前关键字右边指针所指子树中“最左下”的元素

终端结点关键字

删除后结点关键字个数未低于下限，无需任何处理

低于下限

右兄弟够借，则用当前结点的后继、后继的后继依次顶替空缺

左兄弟够借，则用当前结点的前驱、前驱的前驱依次顶替空缺

左(右)兄弟都不够借，则需要与父结点内的关键字、左(右)兄弟进行合并。合并后导致父节点关键字数量-1，可能需要继续合并。



1. 下列关于B树性质的描述, 错误的是:

- A) 所有叶子节点都在同一层
- B) 每个节点的关键字数量有上下界限制
- C) 关键字在节点内是有序排列的
- D) 插入操作可以在任何层次的节点进行

正确答案: D

2. 在一个m阶B树中, 非根节点的关键字数量范围是:

- A) $\lceil m/2 \rceil - 1$ 到 $m-1$
- B) $\lceil m/2 \rceil$ 到 m
- C) $\lceil m/2 \rceil - 1$ 到 m
- D) $\lceil m/2 \rceil$ 到 $m-1$

正确答案: A



1. 在一个5阶B树中, 根节点最少有 _____ 个关键字, 除根节点外, 每个节点最少有 _____ 个关键字, 最多有 _____ 个关键字。每个节点最多有 _____ 个子节点。

正确答案: 1、2、4、5

解析: 5阶B树($m=5$):

- 根节点最少关键字数: 1 (当树非空时)
- 非根节点最少关键字数: $\lceil m/2 \rceil - 1 = \lceil 5/2 \rceil - 1 = 3 - 1 = 2$
- 最多关键字数: $m - 1 = 4$
- 最多子节点数: $m = 5$

2. 当向B树插入关键字导致节点关键字数超过上限时, 需要进行 _____ 操作, 该操作会将节点的 _____ 个关键字提升到父节点, 并将原节点分裂为两个节点。

正确答案: 分裂、中间 (或第 $\lceil m/2 \rceil$ 个)

解析: B树插入时, 如果节点关键字数达到 m 个 (超过 $m-1$ 的限制), 需要进行分裂操作。将中间关键字 (第 $\lceil m/2 \rceil$ 个) 提升到父节点, 原节点分裂为两个各含 $\lceil m/2 \rceil - 1$ 个关键字的节点。



已知一个5阶B树，树高为4（根节点高度为1）：

a) 该B树最多能容纳多少个关键字？

b) 该B树最少能容纳多少个关键字？

c) 如果该B树有10万个关键字，树高最少是多少？

a) 最多关键字数（每个节点都填满）：

- 第1层（根）：1个节点，最多4个关键字
- 第2层：最多5个子节点，每个最多4个关键字 $\rightarrow 5 \times 4 = 20$
- 第3层：每个第2层节点最多有5个子节点，所以第3层最多有 $5 \times 5 = 25$ 个节点，每个最多4个关键字 $\rightarrow 25 \times 4 = 100$
- 第4层：每个第3层节点最多有5个子节点，所以第4层最多有 $25 \times 5 = 125$ 个节点，每个最多4个关键字 $\rightarrow 125 \times 4 = 500$
- 总最多关键字数 = $4 + 20 + 100 + 500 = 624$

b) 最少关键字数（每个节点都只有最少关键字）：

- 第1层（根）：最少1个关键字
- 第2层：根节点最少有2个子节点，每个最少2个关键字 $\rightarrow 2 \times 2 = 4$
- 第3层：每个第2层节点最少有3个子节点，第3层最少有 $2 \times 3 = 6$ 个节点，每个最少2个关键字 $\rightarrow 6 \times 2 = 12$
- 第4层：每个第3层节点最少有3个子节点，第4层最少有 $6 \times 3 = 18$ 个节点，每个最少2个关键字 $\rightarrow 18 \times 2 = 36$
- 总最少关键字数 = $1 + 4 + 12 + 36 = 53$

通用公式（满树时）：

$$\begin{aligned}\text{总关键字数} &= 4 \times (1 + 5 + 5^2 + \dots + 5^{h-1}) \\ &= 4 \times \frac{5^h - 1}{5 - 1} \\ &= 4 \times \frac{5^h - 1}{4} \\ &= 5^h - 1\end{aligned}$$

求解h：

我们需要 $5^h - 1 \geq 100,000$

即 $5^h \geq 100,001$

计算：

- $5^7 = 78,125$ （不够）
- $5^8 = 390,625$ （足够）

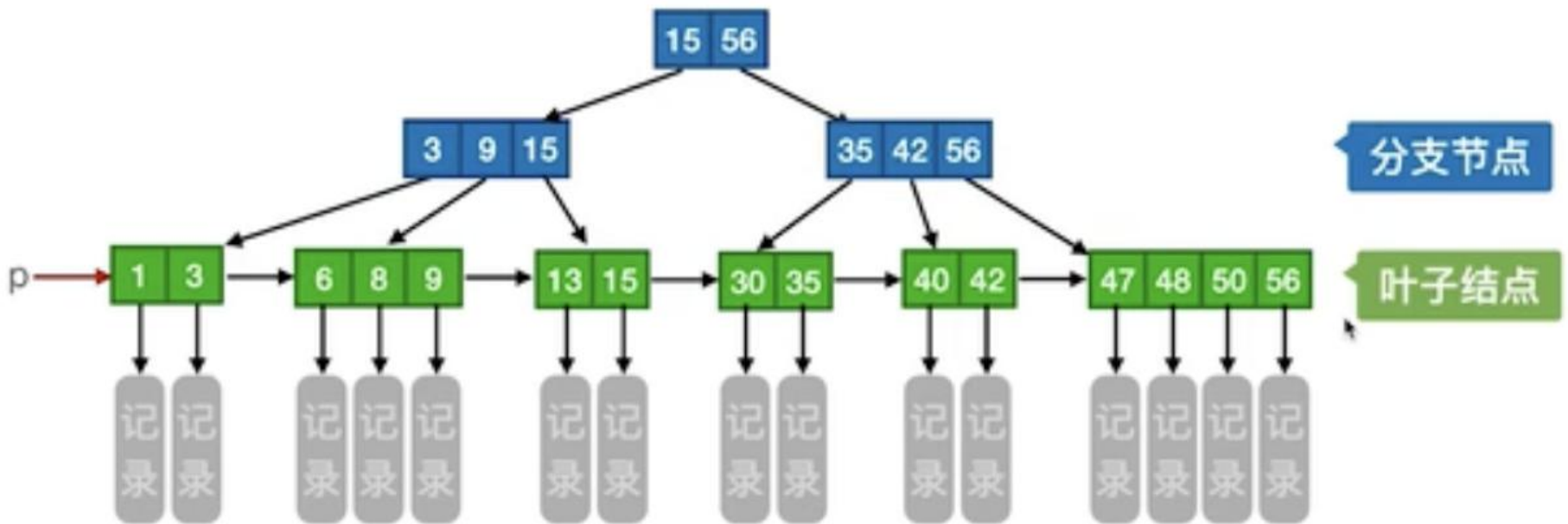
$\therefore h = 8$



B+树



- B+ 树是B 树的变种，它与B 树的不同之处在于：
 - ✓ 所有关键码都存放在叶结点中，上层的非叶结点的关键码是其子树中最小（或最大）关键码的复写。
 - ✓ 叶结点包含了全部关键码及指向相应数据记录存放地址的指针，且叶结点本身按关键码从小到大顺序链接。
- 每个非叶结点结构有两种方式处理。按下层结点“最大关键码复写”和“最小关键码复写”。



注：以上是一棵4阶B+树



按“最大关键码复写”原则组织



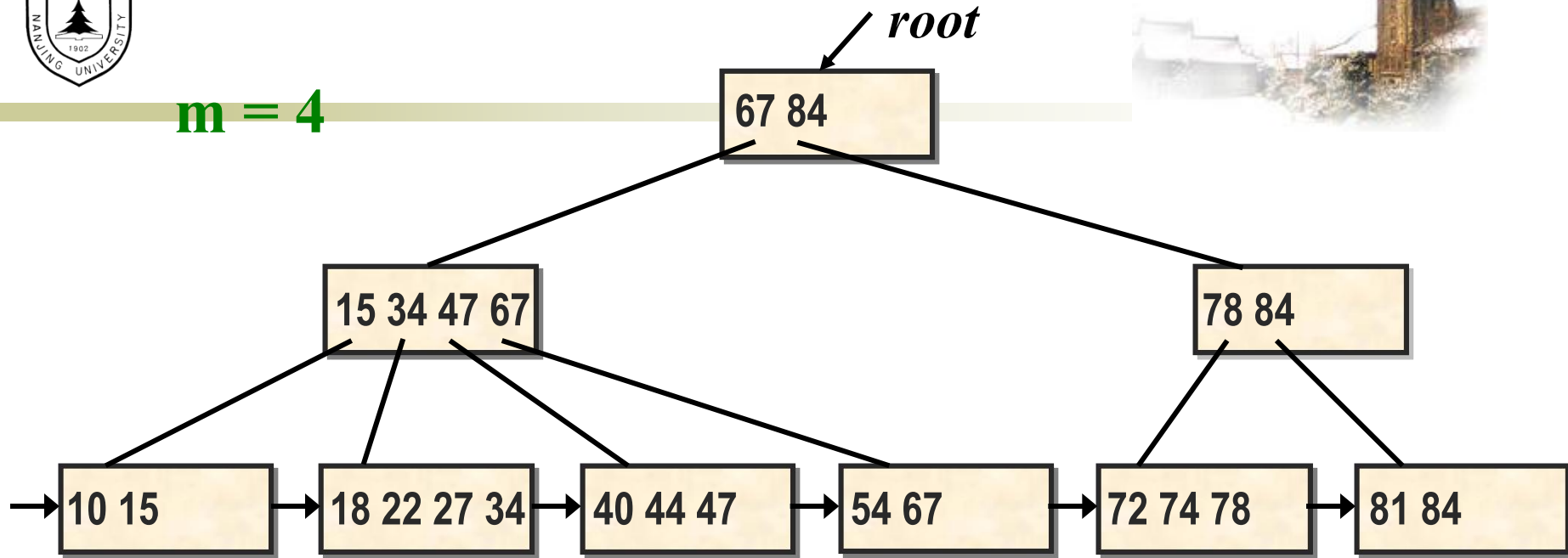
- 一棵 m 阶B+ 树的结构定义如下：
 - ◆ 每个结点最多有 m 棵子树；
 - ◆ 根结点最少有 1 棵子树，除根结点外，其他结点至少有 $\lceil m/2 \rceil$ 棵子树；
 - ◆ 有 n 棵子树的结点有 n 个关键码。
 - ◆ 所有非叶结点可以看成是叶结点的索引，结点中关键码 K_i 与指向子树的指针 P_i 构成对子树 (即下一层索引块) 的索引项 (K_i, P_i) ， K_i 是子树中最大的关键码。



- ◆ 所有叶结点在同一层，按**从小到大**的顺序存放全部关键码，各个叶结点顺序链接。
- 叶结点中存放的是对实际数据记录的索引，每个索引项 (K_i, P_i) 给出数据记录的关键码及实际存储地址。
- 例如，在一棵4阶B+ 树中，所有非叶结点中的子树棵数 $2 \leq n \leq 4$ ，其所有的关键码都出现在叶结点中，且在叶结点中关键码**有序**地排列。上面各层结点中的关键码都是其子树上最大关键码的副本。



$m = 4$



- 通常在B+ 树中有两个头指针：一个指向B+ 树的根结点，一个指向关键码最小的叶结点。因此，可以对B+ 树进行两种搜索运算：
 - 循叶结点自己拉起的链表顺序搜索；
 - 从根开始进行自顶向下直到叶结点的随机搜索。



B+ 树的插入



- B+ 树的插入仅在叶结点上进行。每插入一个(关键码-指针)索引项后都要判断结点中的索引项个数是否超出范围 m 。
- 当插入后叶结点中的关键码个数 $n > m$ 时，需要将叶结点分裂为两个结点：它们包含的关键码个数分别为 $\lceil (m+1)/2 \rceil$ 和 $\lfloor (m+1)/2 \rfloor$ 。并且它们的双亲结点中应同时包含这两个结点的最大关键码和结点地址。
- 在非叶结点中关键码的插入与叶结点的插入情况类似，但在做根结点分裂时，必须创建新的父结点，作为树的新根。

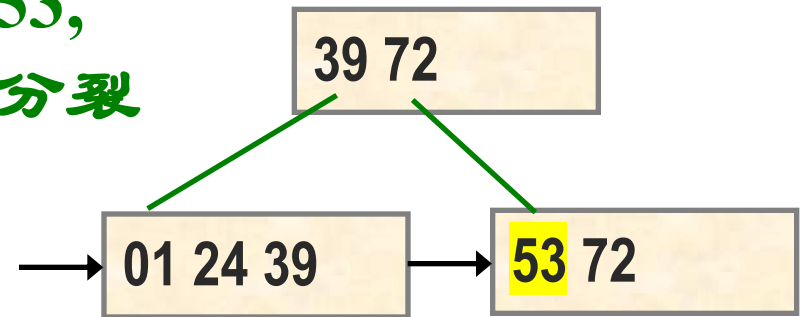


例如，在一棵4阶B+树中的插入过程如下。

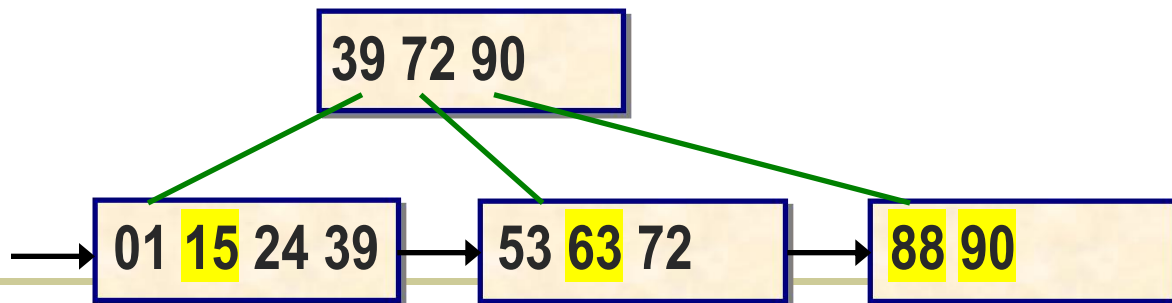
连续插入24, 72, 01, 39的B+树



加入53, 结点分裂

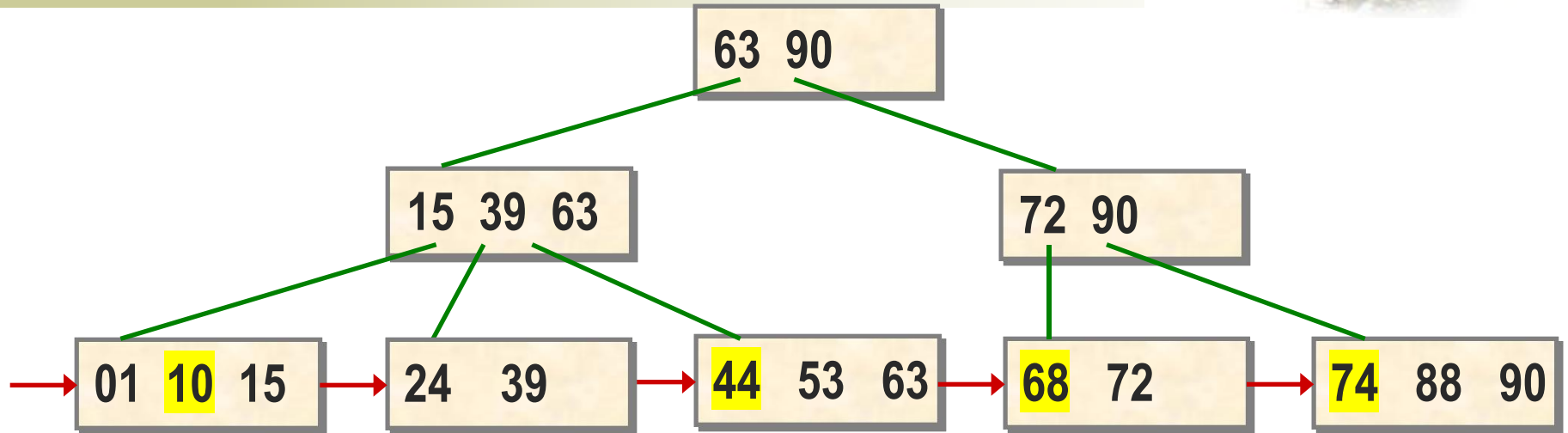


加入63, 90, 88, 15的B+树





加入10, 44, 68, 74的B+树





B+树的删除



- B+ 树的删除仅在叶结点上进行。
- 当在叶结点上删除一个(关键码-指针)索引项后，结点中的索引项个数仍然不少于 $\lceil m/2 \rceil$ ，这属于简单删除，其上层索引可以不改变。
 - 如果删除结点的最大关键码，但因在其上层的副本只起了一个引导搜索的“分界关键码”的作用，所以即使树中已经删除了关键码，但上层的副本仍然可以保留。
- 如果在叶结点中删除后，结点中索引项个数小于 $\lceil m/2 \rceil$ ，必须做结点的调整或合并工作。如果右兄弟结点中的子树棵数大于 $\lceil m/2 \rceil$ ，从右兄弟结点中移最左的（关键码-指针）索引项到这个被删关键码所在的结点，并修改上层的“分界关键码”的值。



$m = 4$



67 84

15 34 47 67

78 84

→ 10 15

→ 18 22 27 34

→ 40 44 47

×

→ 54 67

→ 72 74 78

→ 81 84

简单删除关键码47,
上层索引可以不改

67 84

15 34 47 67

78 84

→ 10 15

→ 18 22 27 34

→ 40 44

→ 54 67

→ 72 74 78

→ 81 84

从4阶B+树中做简单删除



$m = 4$



67 84

15 34 47 67

78 84

→ 10 15

X

→ 18 22 27 34

→ 40 44 47

→ 54 67

→ 72 74 78

→ 81 84

删除关键码15, 调整
上层索引改变

67 84

18 34 47 67

78 84

→ 10 18

→ 22 27 34

→ 40 44 47

→ 54 67

→ 72 74 78

→ 81 84

从4阶B+树中删除时的调整



- 如果右兄弟结点的关键码数已达到下限 $\lceil m/2 \rceil$, 没有多余的关键码可以移入被删关键码所在的结点, 必须进行结点的合并。将右兄弟结点中的所有(关键码-指针)索引项移入被删关键码所在结点, 再将右兄弟结点删去。
- 这种结点合并将导致双亲结点中“分界关键码”的减少, 有可能减到非叶结点中关键码个数的下限 $\lceil m/2 \rceil$ 以下。这样将引起双亲结点的调整或合并。
- 如果根结点的最后两个子女结点合并, 树的层数就会减少一层。



67 84

18 34 47 67

78 84

→ 10 18

→ 22 27 34

→ 40 44 47

→ 54 67

→ 72 74 78

→ 81 84

× ×

删除关键码74,
78, 结点合并

47 84

18 34 47

67 84

→ 10 18

→ 22 27 34

→ 40 44 47

→ 54 67

→ 72 81 84



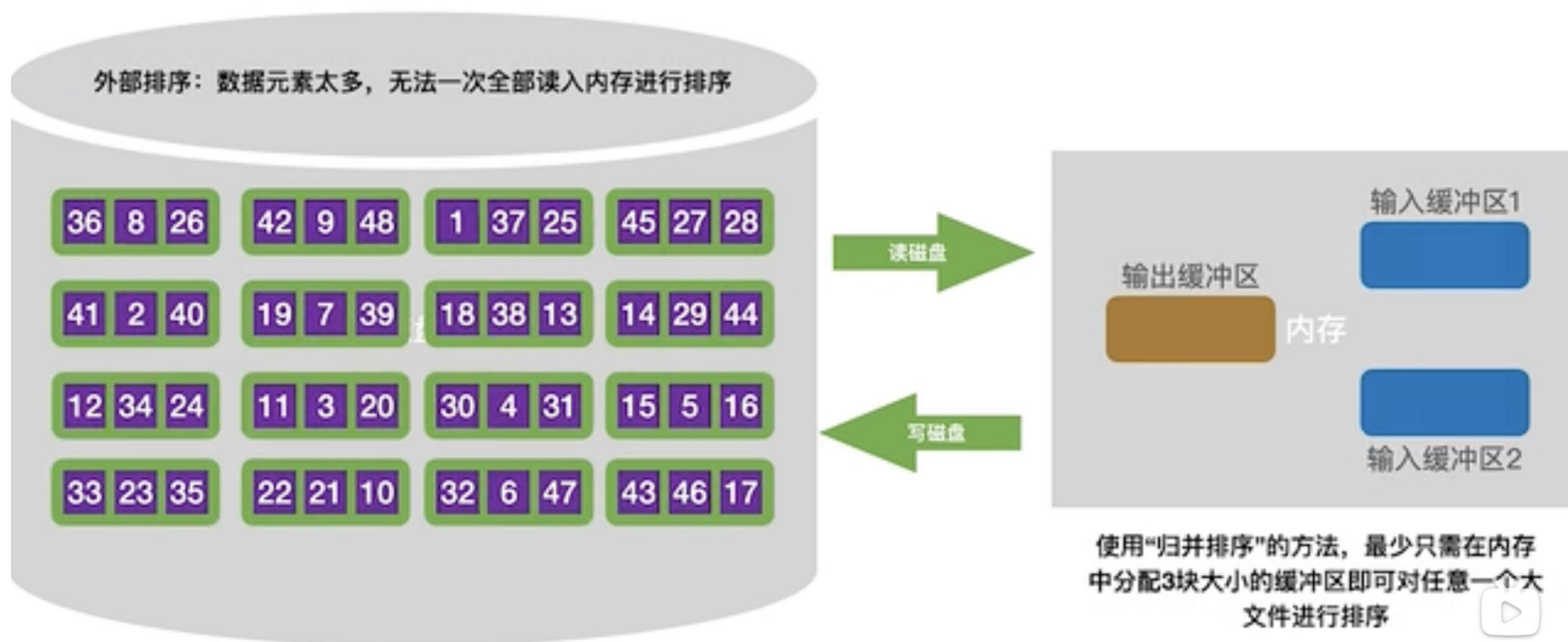
	m阶B树	m阶B+树
类比	二叉查找树的进化——>m叉查找树	分块查找的进化——>多级分块查找
关键字与分叉	n个关键字对应n+1个分叉（子树）	n个关键字对应n个分叉
结点包含的信息	所有结点中都包含记录的信息	只有最下层叶子结点才包含记录的信息 (可使树更矮)
查找方式	不支持顺序查找。查找成功时，可能停在任何一层结点，查找速度“不稳定”	支持顺序查找。查找成功或失败都会到达最下一层结点，查找速度“稳定”
相同点	除根节点外，最少 $\lceil m/2 \rceil$ 个分叉（确保结点不要太“空”） 任何一个结点的子树都要一样高（确保“绝对平衡”）	



10.3 外排序



- 当待排序的记录数目特别多时，在内存中不能一次处理。必须把它们以文件的形式存放于外存，排序时再把它们一部分一部分调入内存进行处理。这样，在排序过程中必须不断地在内存与外存之间传送数据。
- 这种基于外部存储设备（或文件）的排序技术就是外排序。





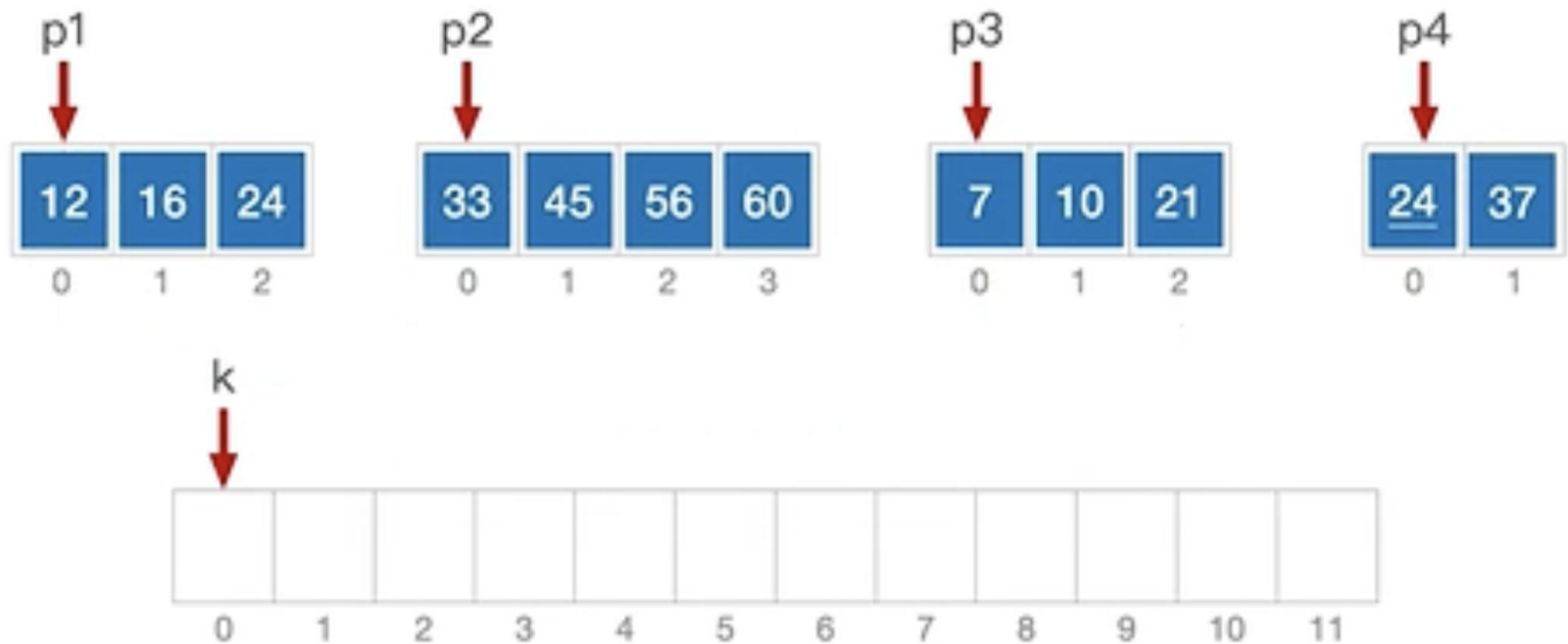
外排序的基本过程



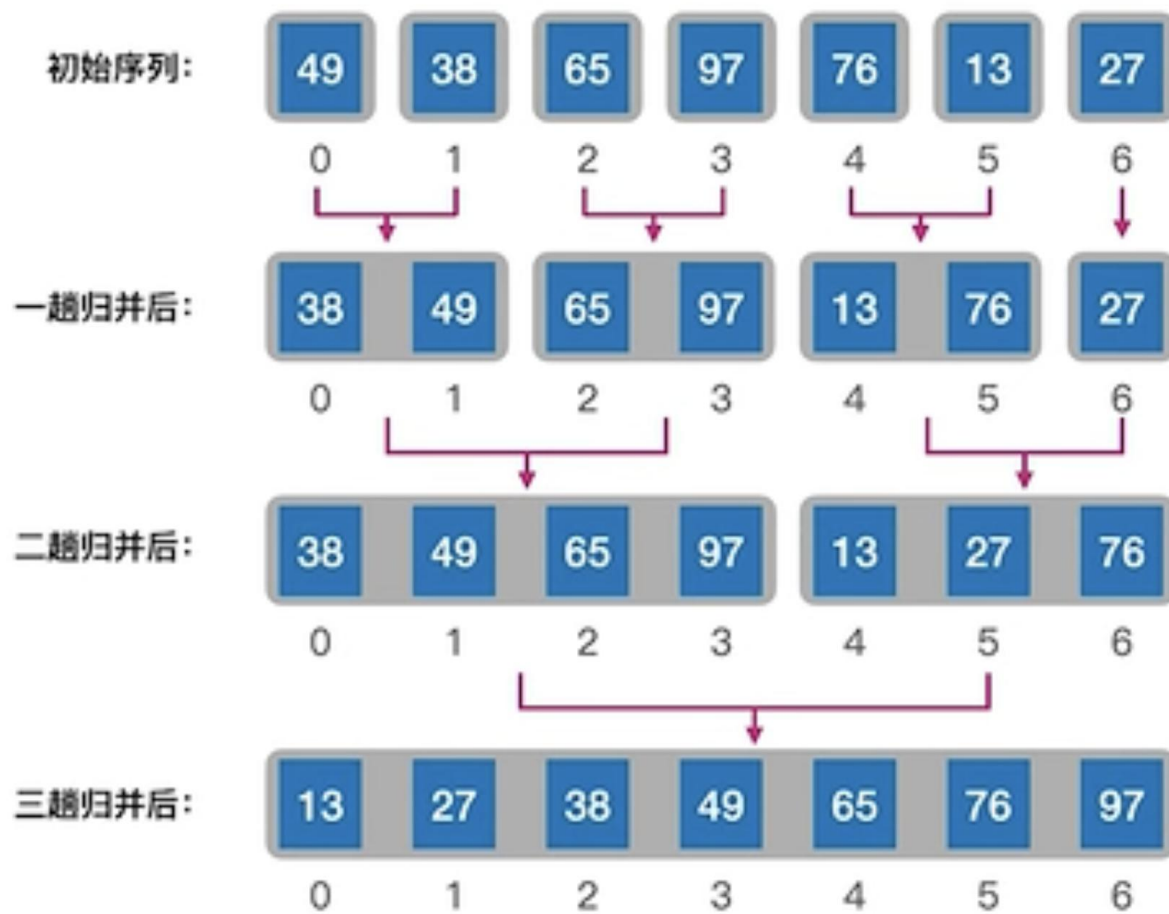
- 基于磁盘进行的排序大多使用**归并排序**方法。其排序过程主要分为两个阶段：
 - ① 建立用于外排序的**内存缓冲区**。根据它们的大小将输入文件划分为若干段,用某种内排序方法对各段进行排序。经过排序的段叫做**初始归并段 (Run)**。当它们生成后,就被写到外存中去。
 - ② 按归并树模式,把①生成的初始归并段加以归并,一趟趟扩大归并段和减少归并段数,直到最后归并成一个大归并段为止。



归并：把两个或多个已经有序的序列合并成一个



"4路"归并——每选出一个元素需对比关键字3次 对比 p1、p2、p3、p4所指元素，选择更小的一个放入 k 所指位置





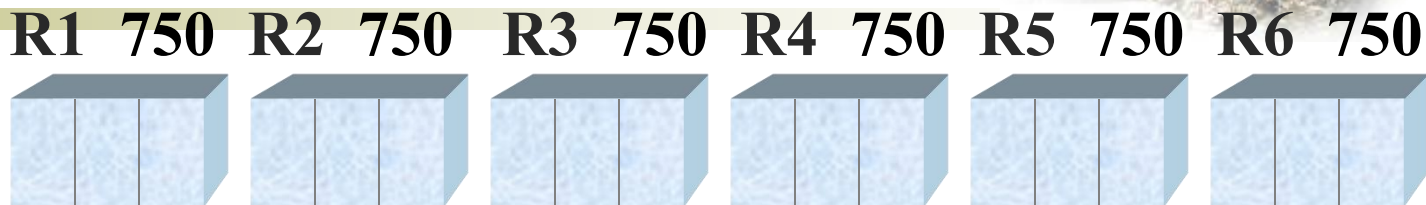
- 示例：设有一个包含**4500**个记录的输入文件。现用一台其内存至多可容纳**750**个记录的计算机对该文件进行排序。输入文件放在磁盘上，磁盘每个页块可容纳**250**个记录，这样全部记录可存储在 $4500 / 250 = 18$ 个页块中。输出文件也放在磁盘上，用以存放归并结果。
- 由于内存中可用于排序的存储区域能容纳**750** 个记录，因此内存中恰好能**存3个页块**的记录。
- 在外排序一开始，把**18块**记录，每**3块**一组，读入内存。利用某种内排序方法进行内排序，形成初始归并段，再写回外存。总共可得到**6个初始归并段**。然后一趟一趟地进行归并排序。



两路归并排序的归并树



初始
归并段



第一趟
归并结果

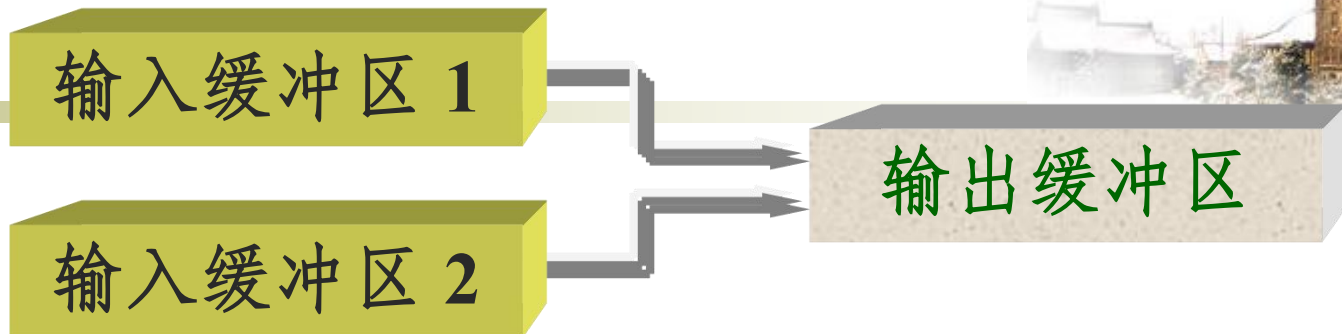


第二趟
归并结果



第三趟
归并结果





- 若把内存区域等份地分为 3 个缓冲区，其中的两个为输入缓冲区，一个为输出缓冲区，可以在内存中利用简单两路归并函数 `merge()` 实现两路归并。
- 首先，从参加归并排序的两个输入归并段 R_1 和 R_2 中分别读入一块，放在输入缓冲区 1 和输入缓冲区 2 中。然后在内存中进行两路归并，归并结果顺序存放输出缓冲区中。



- 若总记录个数为 n ，磁盘上每个页块可容纳 b 个记录，内存缓冲区可容纳 i 个页块，则每个初始归并段长度为 $len = i * b$ ，可生成 $m = \lceil n / len \rceil$ 个等长的初始归并段。
- 在做 2 路归并排序时，第一趟从 m 个初始归并段得到 $\lceil m/2 \rceil$ 个归并段，以后各趟将从 l ($l > 1$) 个归并段得到 $\lceil l/2 \rceil$ 个归并段。总归并趟数等于归并树的高度减一： $\lceil \log_2 m \rceil$ 。
- 估计两路归并排序时间 t_{ES} 的上界为：

$$t_{ES} = m * t_{IS} + d * t_{IO} + S * n * t_{mg}$$

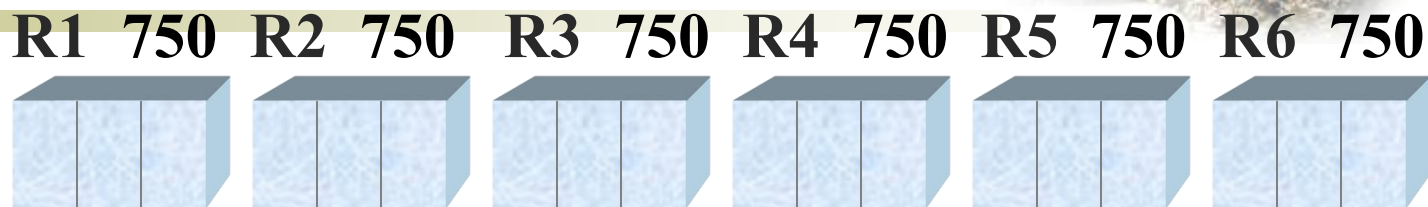
t_{IS} 为每个初始归并段进行内排序的时间； d 为访问外存块的次数； t_{mg} 是取得一个记录的时间； S 是归并趟数



两路归并排序的归并树



初始
归并段



第一趟
归并结果



第二趟
归并结果



第三趟
归并结果





- 对 4500 个记录排序的例子, 各种操作的计算时间如下:
 - ✓ 读 18 个输入块, 内部排序 6 段, 写 18 个输出块
 $= 6 t_{IS} + 36 t_{IO}$
 - ✓ 成对归并初始归并段 $R_1 \sim R_6$
 $= 36 t_{IO} + 4500 t_{mg}$
 - ✓ 归并两个具有 1500 个记录的归并段 R_{12} 和 R_{34}
 $= 24 t_{IO} + 3000 t_{mg}$
 - ✓ 最后将 R_{1234} 和 R_{56} 归并成一个归并段
 $= 36 t_{IO} + 4500 t_{mg}$
- 合计 $t_{ES} = 6 t_{IS} + 132 t_{IO} + 12000 t_{mg}$



- 由于 $t_{IO} = t_{seek} + t_{latency} + t_{rw}$, 其中, t_{seek} 和 $t_{latency}$ 是机械动作, 而 t_{rw} 、 t_{IS} 、 t_{mg} 是电子线路的动作, 所以 t_{IO} 远远大于 t_{IS} 和 t_{mg} 。想要提高外排序的速度, 应着眼于减少 d 。
- 若对相同数目的记录, 在同样页块大小的情况下做 3 路归并或做 6 路归并(当然, 内存缓冲区的数目也要变化), 则可做大致比较:

<u>归并路数 k</u>	<u>总读写磁盘次数 d</u>	<u>归并趟数 S</u>
2	132	3
3	108	2
6	72	1



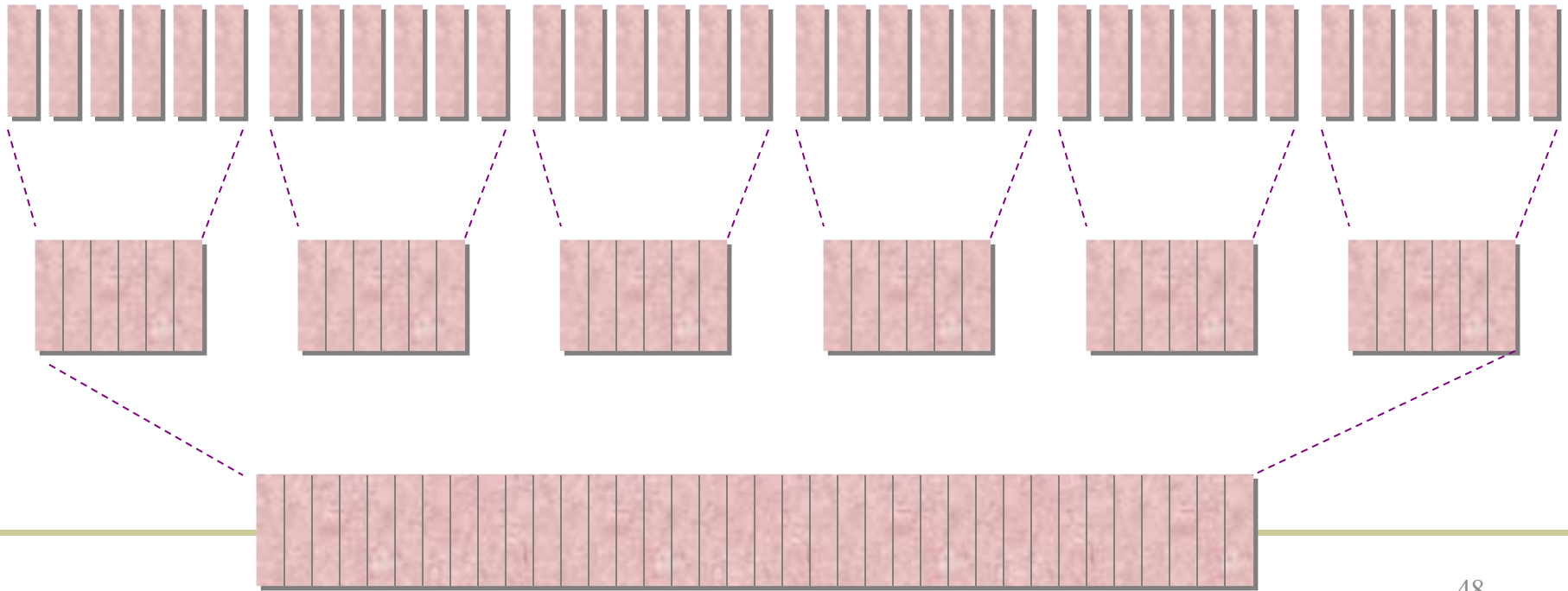
- 增大归并路数,可减少归并趟数,从而减少总读写磁盘次数 d 。
- 对 m 个初始归并段,做 k 路平衡归并,一趟可将 m 个初始归并段归并为 $l = \lceil m/k \rceil$ 个归并段,以后每一趟归并将 l 个归并段归并成 $l = \lceil l/k \rceil$ 个归并段,直到最后形成一个大的归并段为止。归并趟数 $S = \lceil \log_k m \rceil =$ 树的高度减一。



k路平衡归并 (k-way Balanced merging)

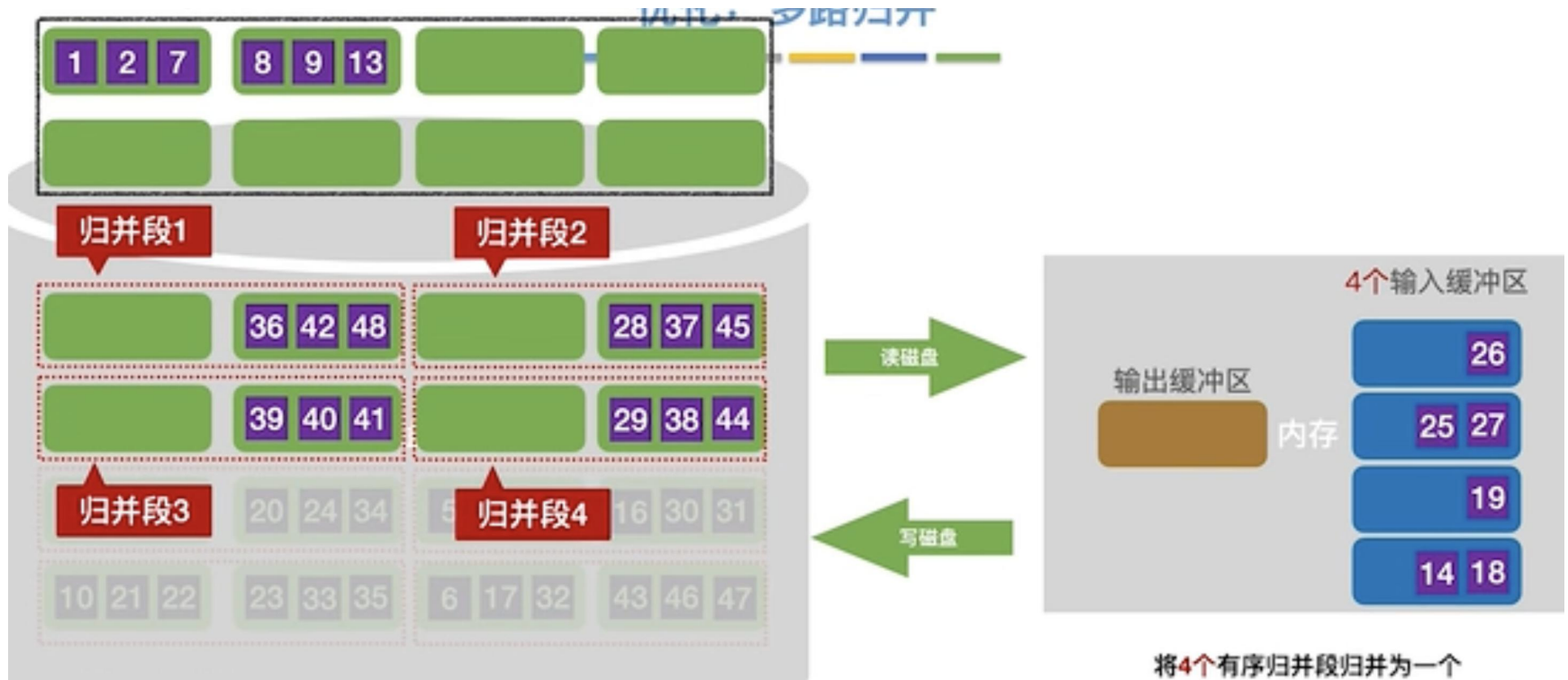


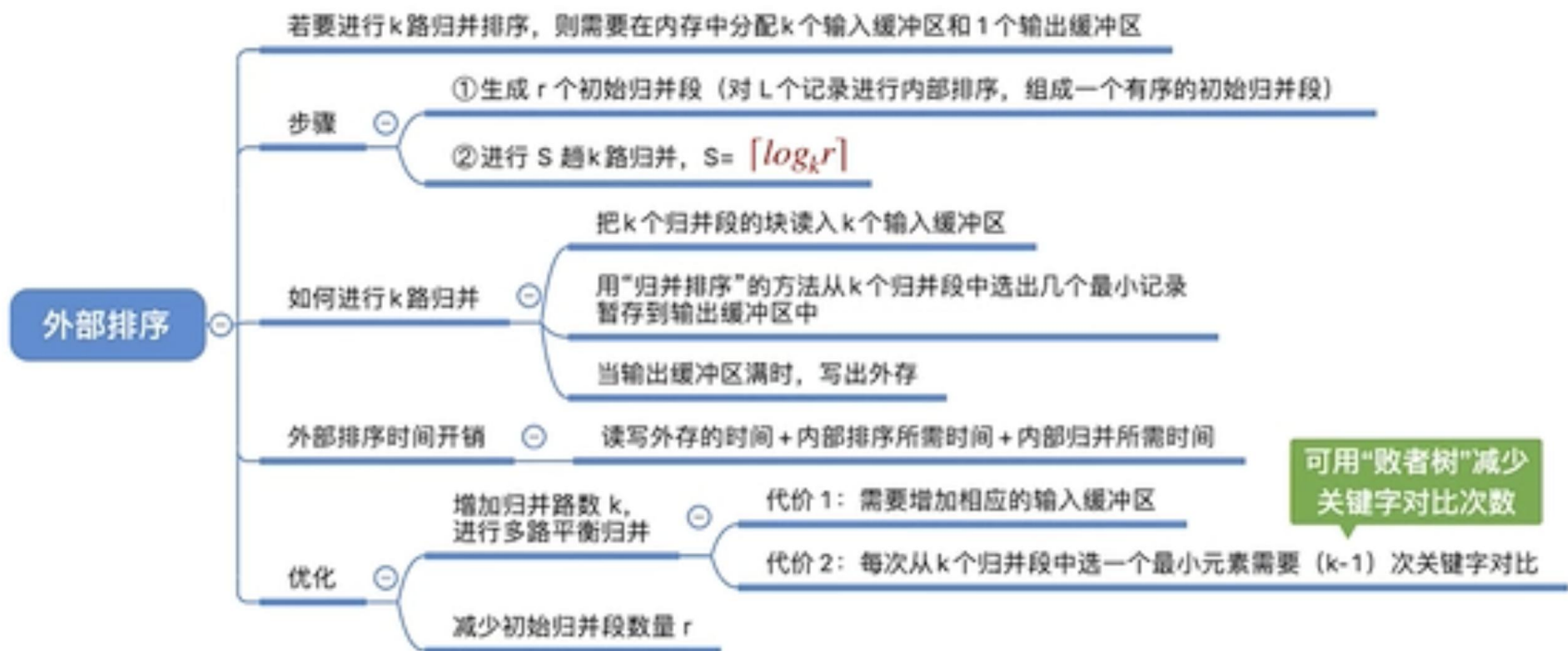
- 做 k 路平衡归并时, 如果有 m 个初始归并段, 则相应的归并树有 $\lceil \log_k m \rceil + 1$ 层, 需要归并 $\lceil \log_k m \rceil$ 趟。下图给出对有 36 个初始归并段的文件做 6 路平衡归并时的归并树。





- 做内部 k 路归并时, 在 k 个记录中选择最小者, 需要顺序比较 $k-1$ 次。每趟归并 n 个记录需要做 $(n-1)*(k-1)$ 次比较, S 趟归并总共需要的比较次数为:
$$S*(n-1)*(k-1) = \lceil \log_k m \rceil * (n-1) * (k-1)$$
$$= \lceil \log_2 m \rceil * (n-1) * (k-1) / \lceil \log_2 k \rceil$$
- 在初始归并段个数 m 与记录个数 n 一定时, $\lceil \log_2 m \rceil * (n-1) = \text{const}$, 而 $(k-1) / \lceil \log_2 k \rceil$ 随着 k 的增大而增大。
- 因此, 增大归并路数 k , 会使得内部归并的时间增大。







- 使用“**败者树**”从 k 个归并段中选最小者, 当 k 较大时 ($k \geq 6$), 选出排序码最小的记录只需比较 $\lceil \log_2 k \rceil$ 次。

$$\begin{aligned} S^*(n-1) * \lceil \log_2 k \rceil &= \lceil \log_k m \rceil * (n-1) * \lceil \log_2 k \rceil \\ &= \lceil \log_2 m \rceil * (n-1) * \lceil \log_2 k \rceil / \lceil \log_2 k \rceil \\ &= \lceil \log_2 m \rceil * (n-1) \end{aligned}$$

- 排序码比较次数与 k 无关, 总的内部归并时间不会随 k 的增大而增大。
- 下面讨论利用败者树在 k 个输入归并段中选择最小者, 实现归并排序的方法。



- 败者树是一棵完全二叉树。其中
 - 每个叶结点存放各归并段在归并过程中当前参加比较的记录；
 - 每个非叶结点存放它两个子女结点中记录**排序码大的结点**(即**败者**)；（假设需要按从小到大排序）
- 在根结点的上一层另外增加一个结点，存放树中当前记录排序码最小的结点(最小记录)。
- **败者树**与**胜者树**的区别在于一个选择了败者(排序码大者)，一个选择了胜者(排序码小者)。



示例：设有 5 个初始归并段，它们中各记录的排序码分别是（增加一个关键码为 ∞ 的记录表示末尾）：

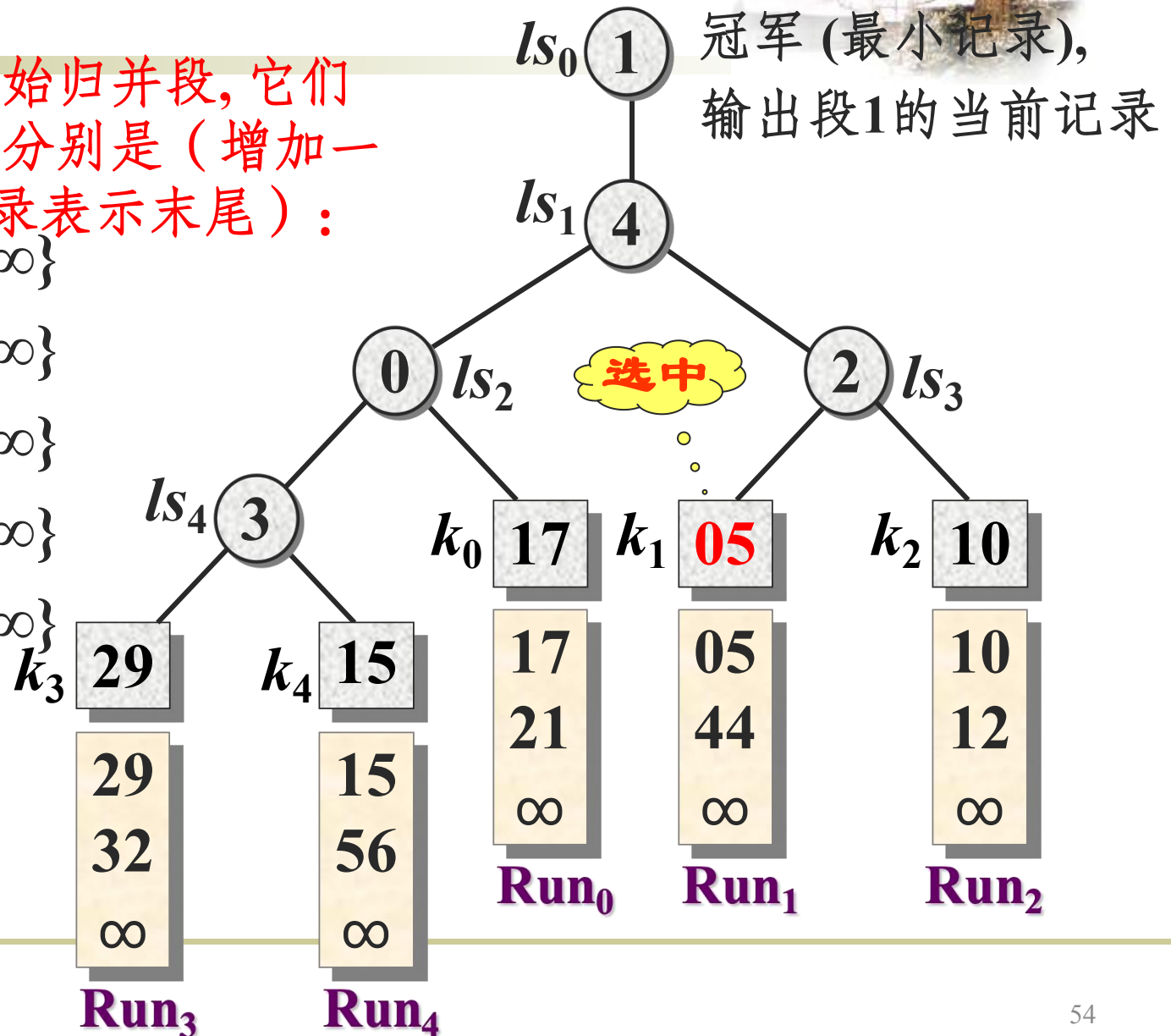
Run0: {17, 21, ∞ }

Run1: {05, 44, ∞ }

Run2: {10, 12, ∞ }

Run3: {29, 32, ∞ }

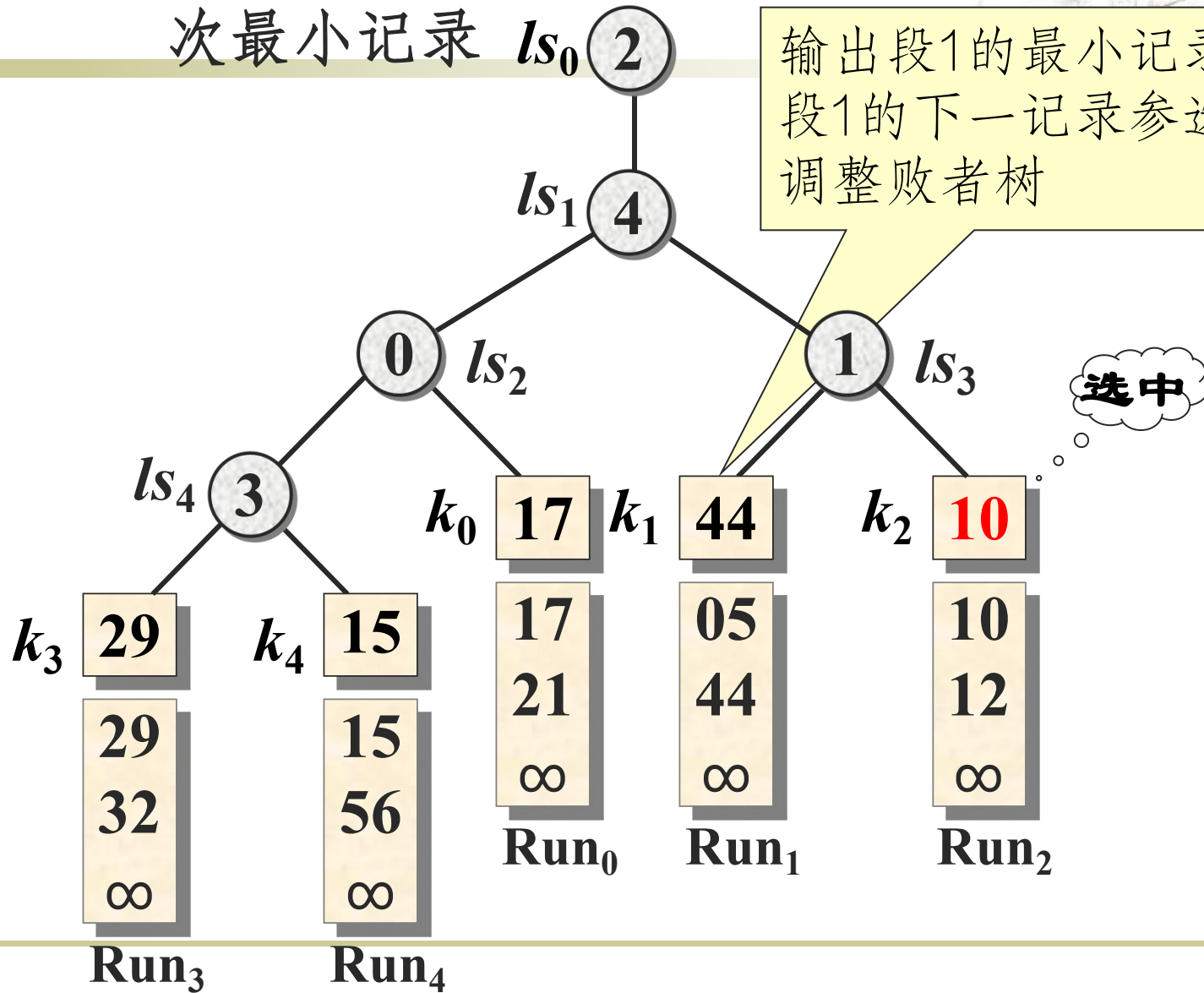
Run4: {15, 56, ∞ }





次最小记录 ls_0 2

输出段1的最小记录，
段1的下一记录参选，
调整败者树





- 败者树的高度为 $\lceil \log_2 k \rceil + 1$ ，在每次调整、找下一个具有最小排序码记录时，最多做 $\lceil \log_2 k \rceil$ 次排序码比较。
- 在内存中应为每一个归并段分配一个输入缓冲区，其大小应能容纳一个页块的记录，编号与归并段号一致。每个输入缓冲区应有一个指针，指示当前参加归并的记录。
- 在内存中还应设立一个输出缓冲区，其大小相当于一个页块大小。它也有一个缓冲区指针，指示当前可存放结果记录的位置。每当一个记录 i 被选出，就执行 **OutputRecord(i)** 操作，将记录存放到输出缓冲区中。



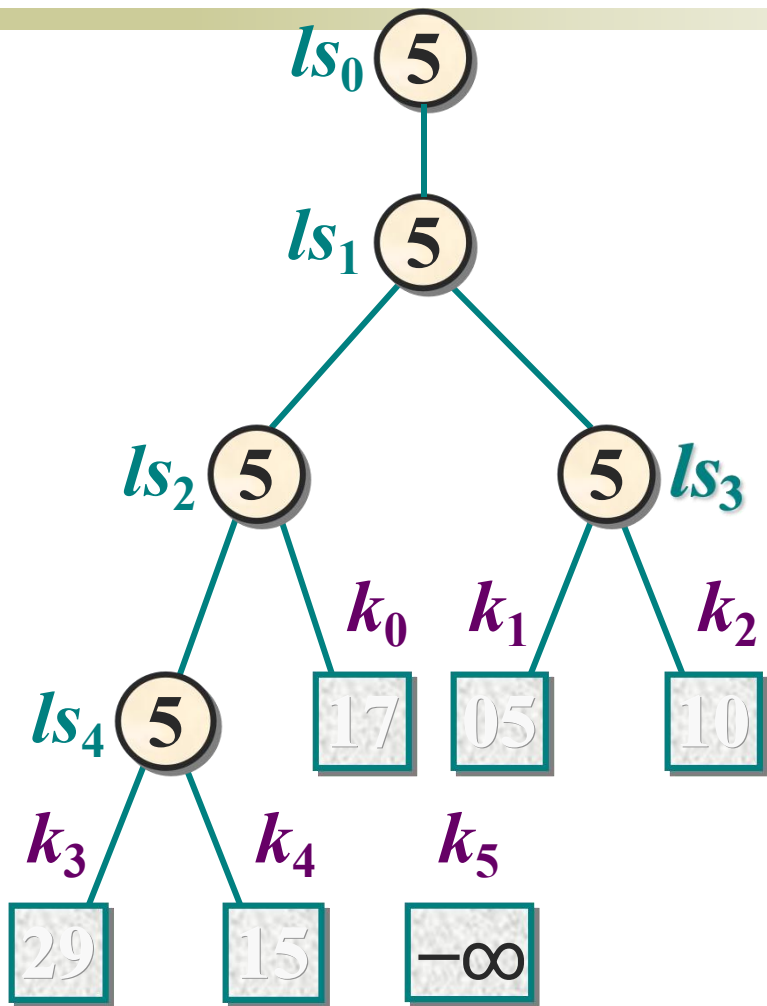
- 在实现利用败者树进行多路平衡归并算法时,把败者树的叶结点和非叶结点分开定义。
- 败者树叶结点 $\text{key}[]$ 有 $k+1$ 个, $\text{key}[0]$ 到 $\text{key}[k-1]$ 存放各归并段当前参加归并的记录的排序码, $\text{key}[k]$ 是辅助工作单元,在初始建立败者树时使用:存放一个最小的在各归并段中不可能出现的排序码: $-\text{MaxValue}$ 。
- 败者树非叶结点 $\text{loser}[]$ 有 k 个,其中 $\text{loser}[1]$ 到 $\text{loser}[k-1]$ 存放各次比较的败者的归并段号, $\text{loser}[0]$ 中是最后胜者所在归并段号。另外还有一个存放各归并段参加归并记录的数组 $r[k]$ 。



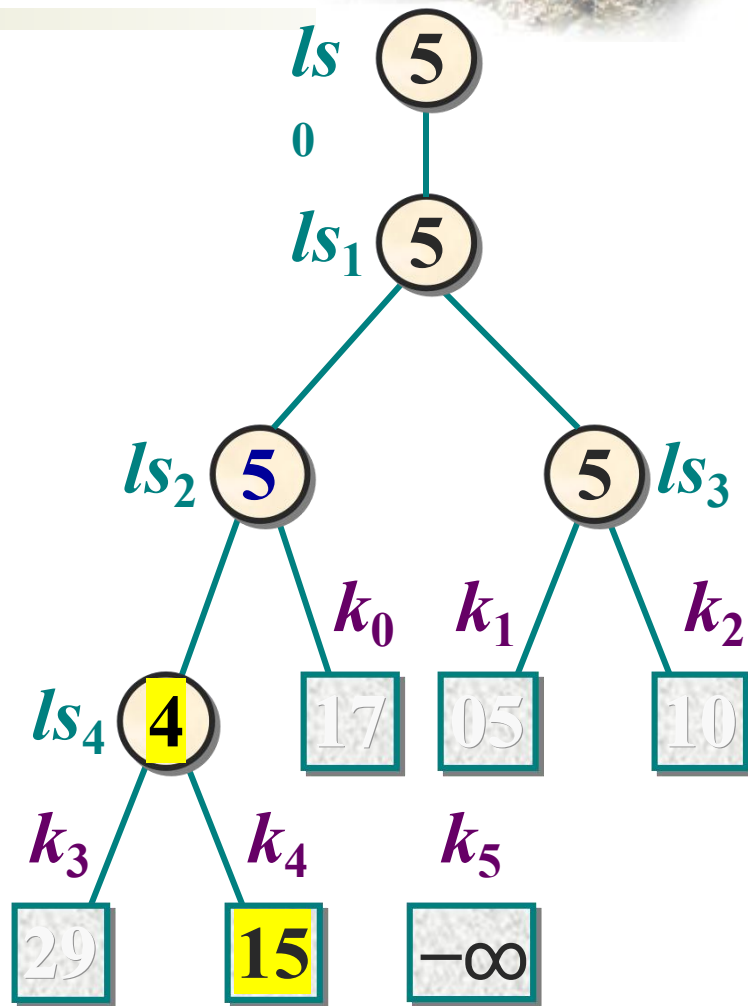
- 每选出一个当前排序码最小的记录,就需要在将它送入输出缓冲区之后,从相应归并段的输入缓冲区中取出下一个参加归并的记录,替换已经取走的最小记录,再从叶结点到根结点,沿某一特定路径进行调整,将下一个排序码最小记录的归并段号调整到`loser[0]`中。
- 段结束标志`MaxNum`升入`loser[0]`,排序完成。



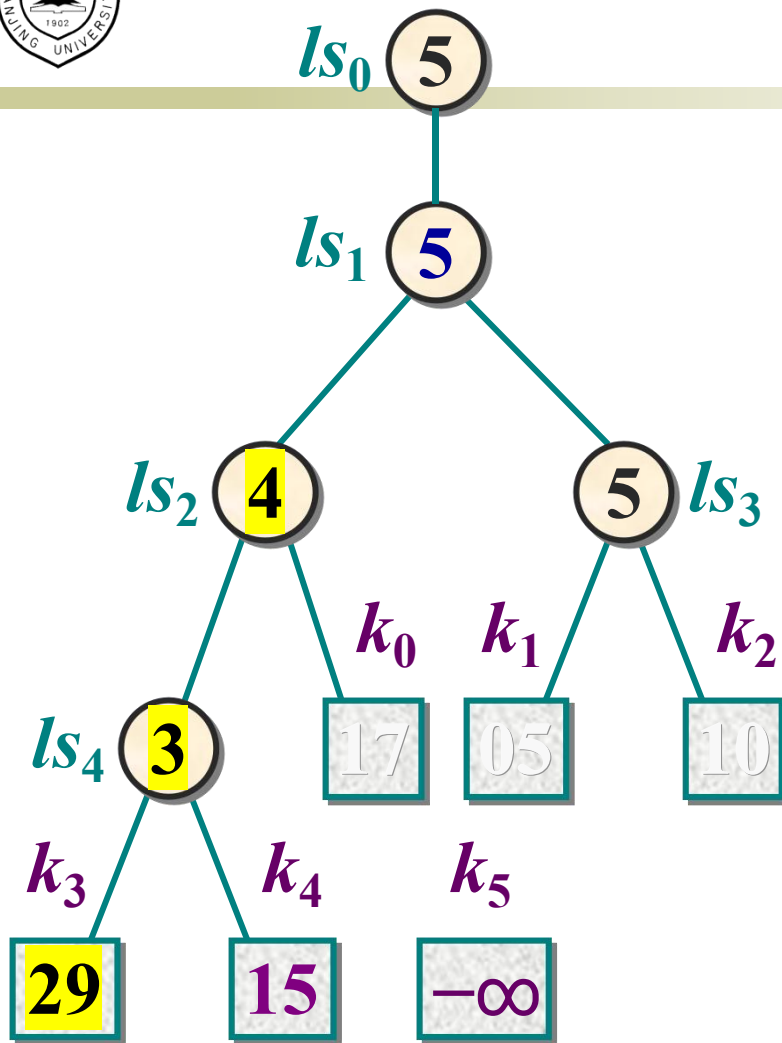
利用败者树进行 5 路平衡归并的过程



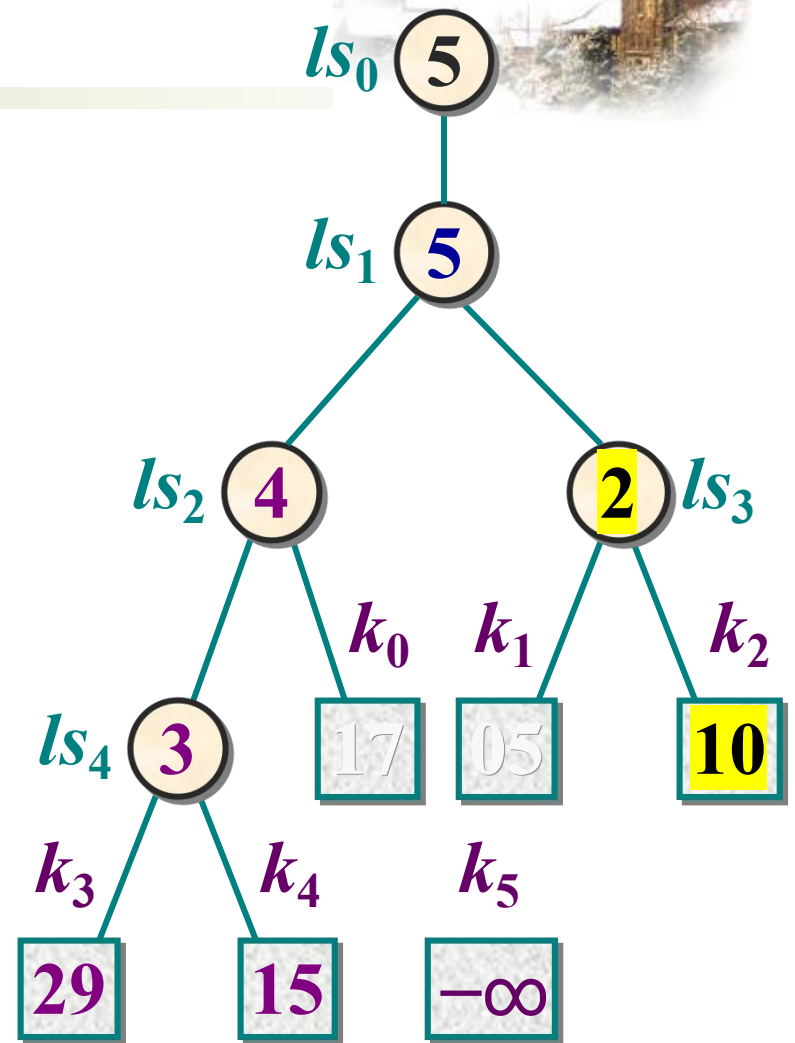
(1) 初始状态



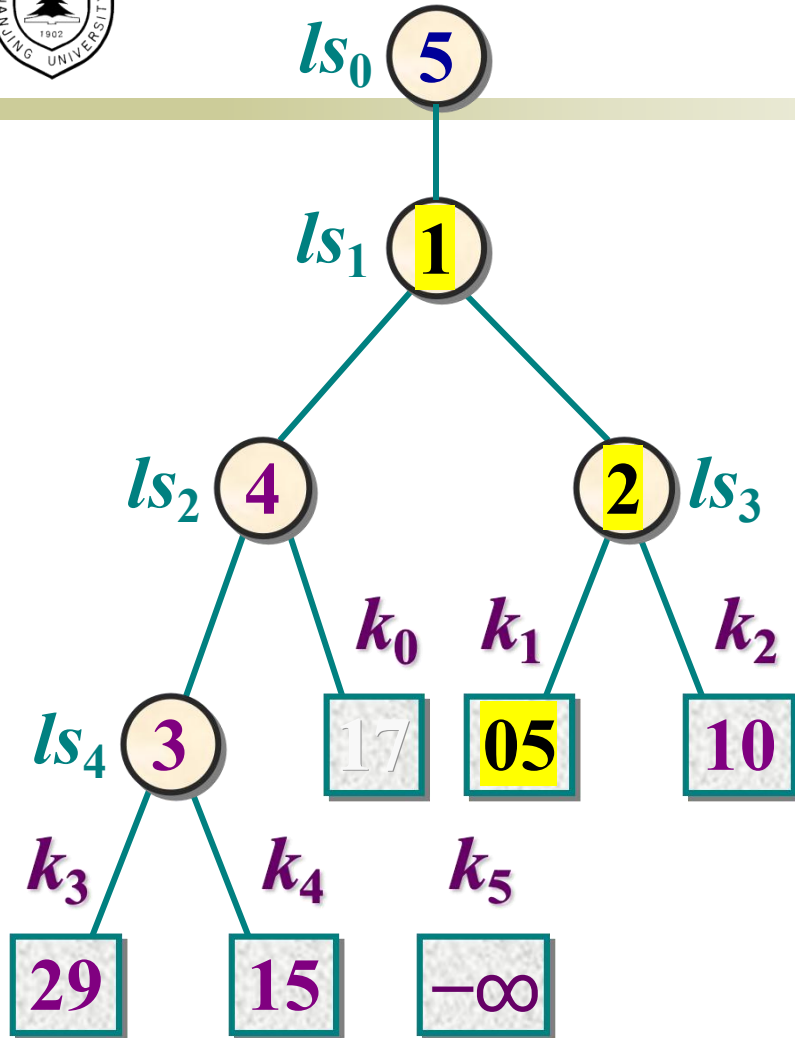
(2) 加入15, 调整



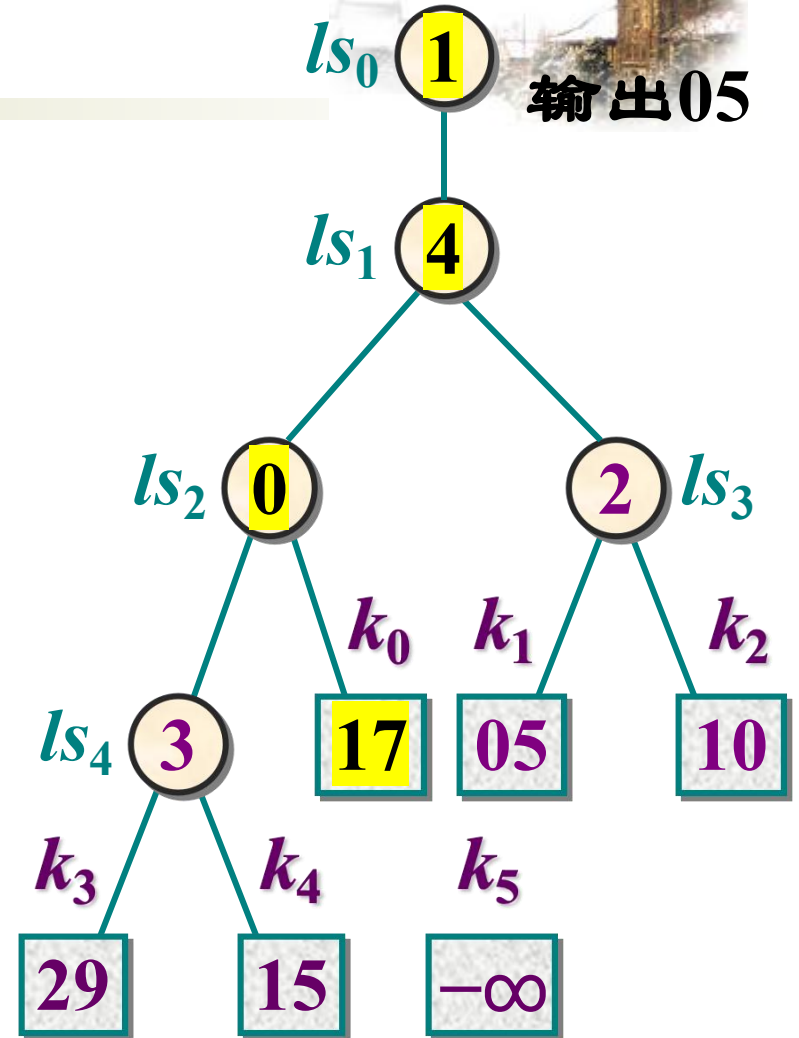
(3) 加入29, 调整



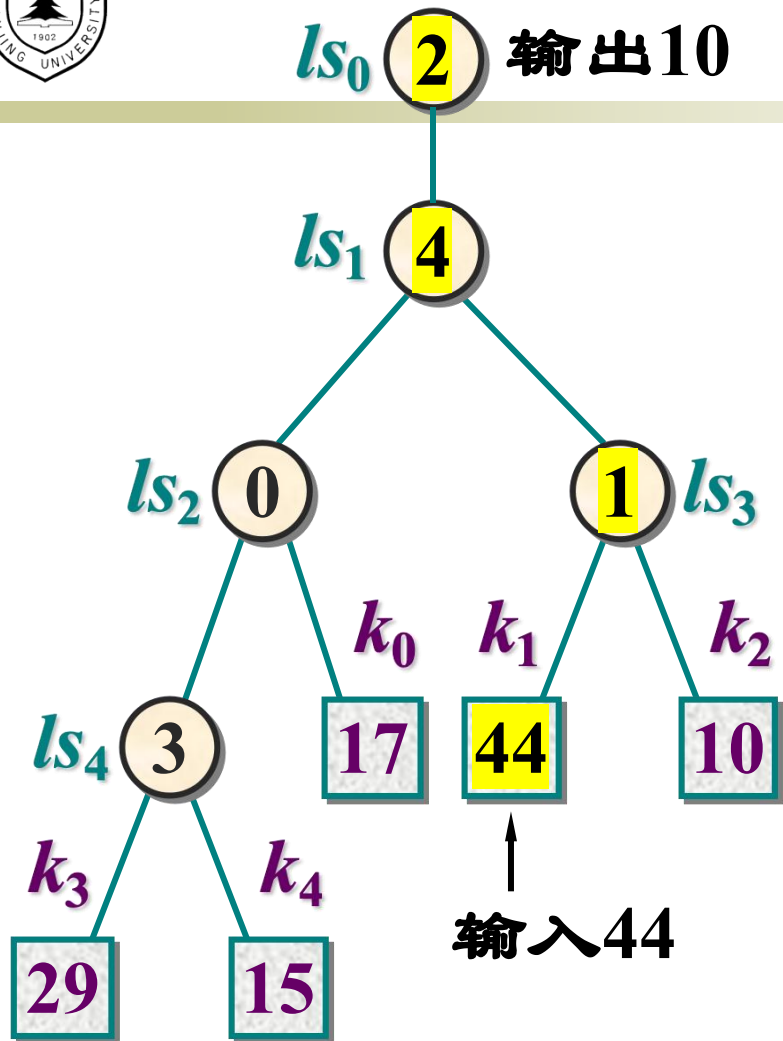
(4) 加入10, 调整



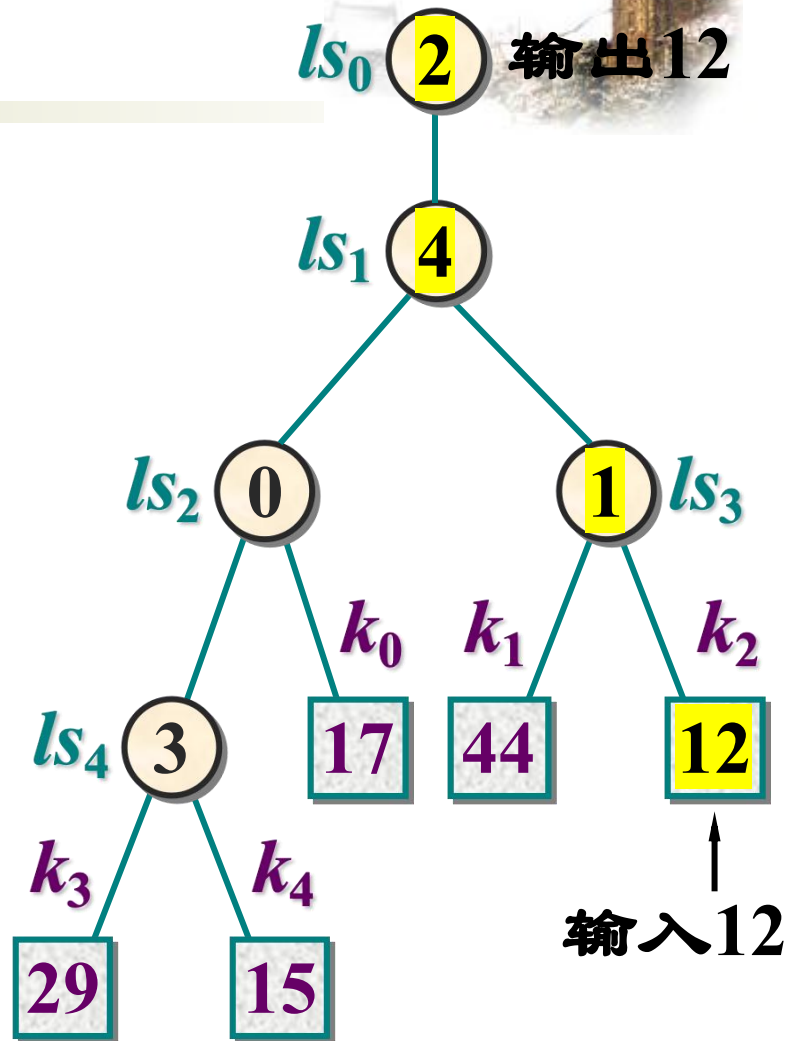
(5) 加入05, 调整



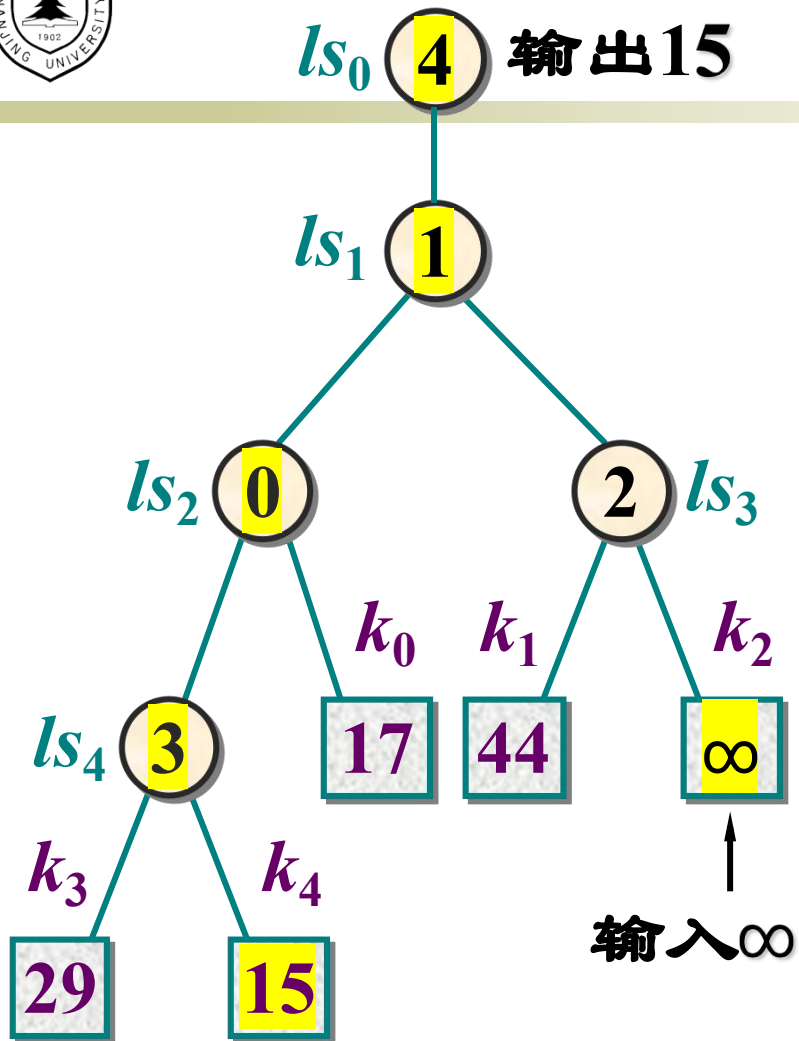
(6) 加入17, 调整



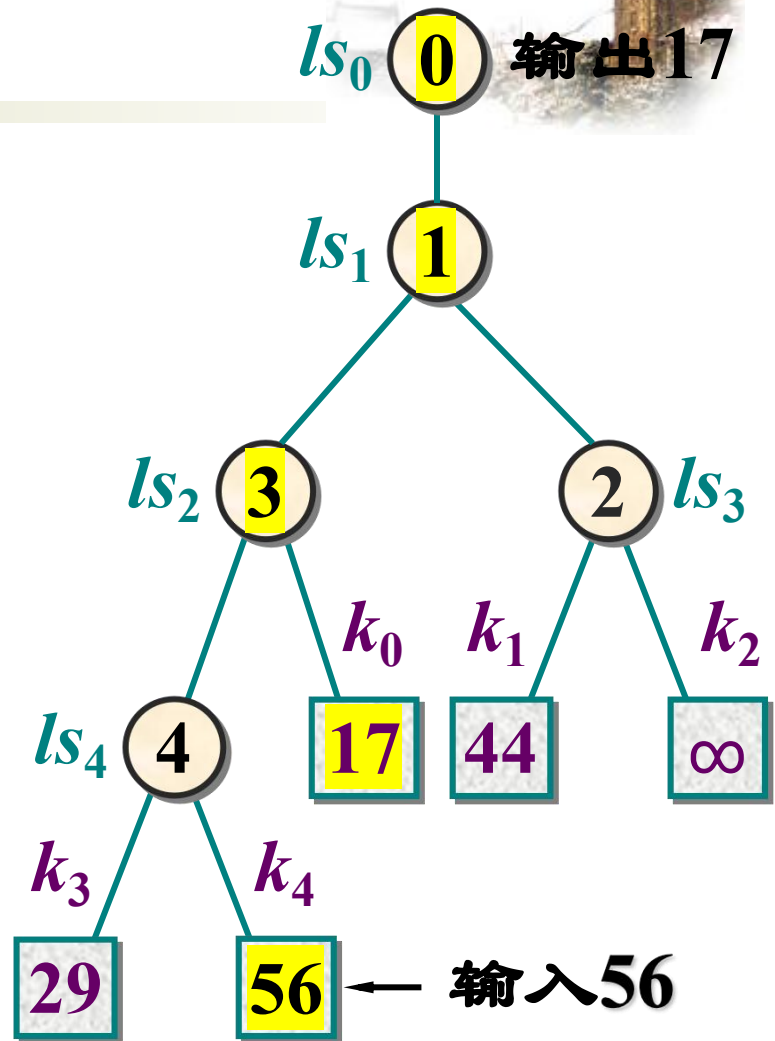
(7) 输出05后调整



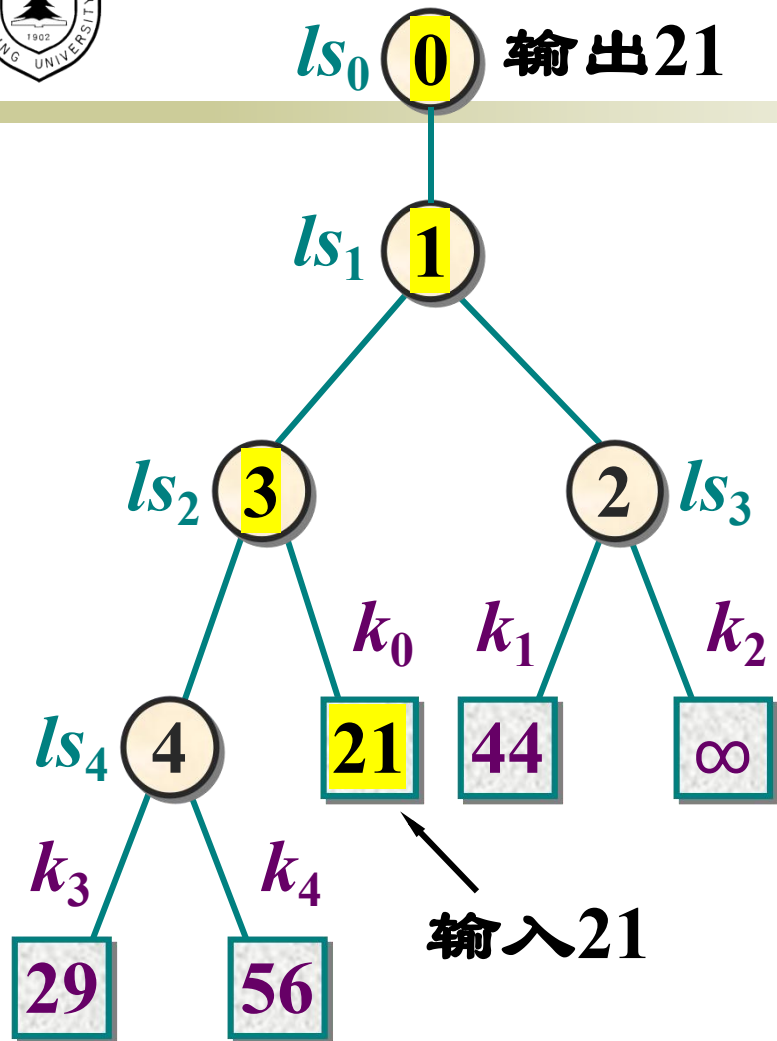
(8) 输出10后调整



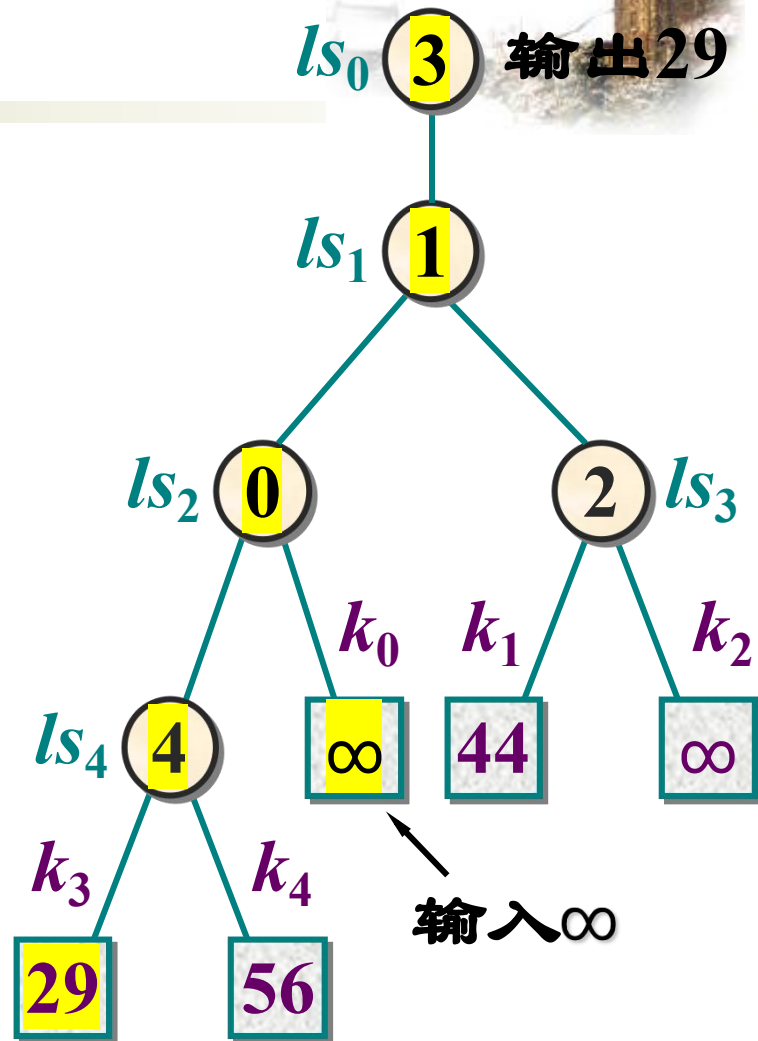
(9) 输出12后调整



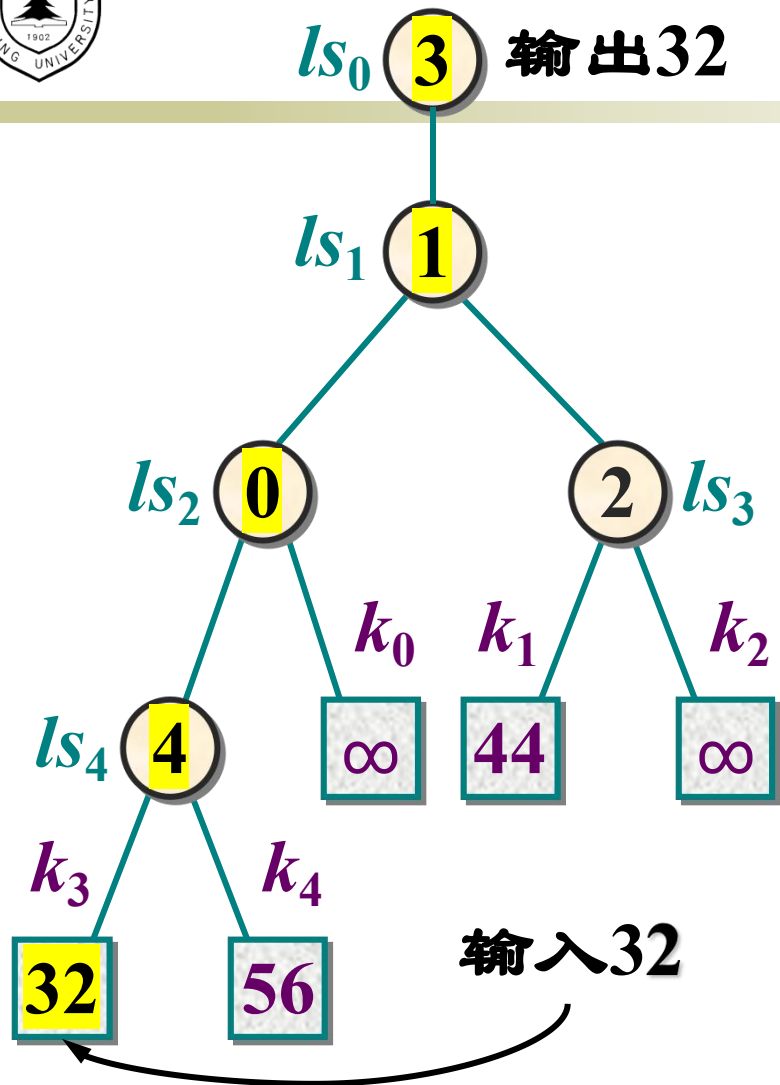
(10) 输出15后调整



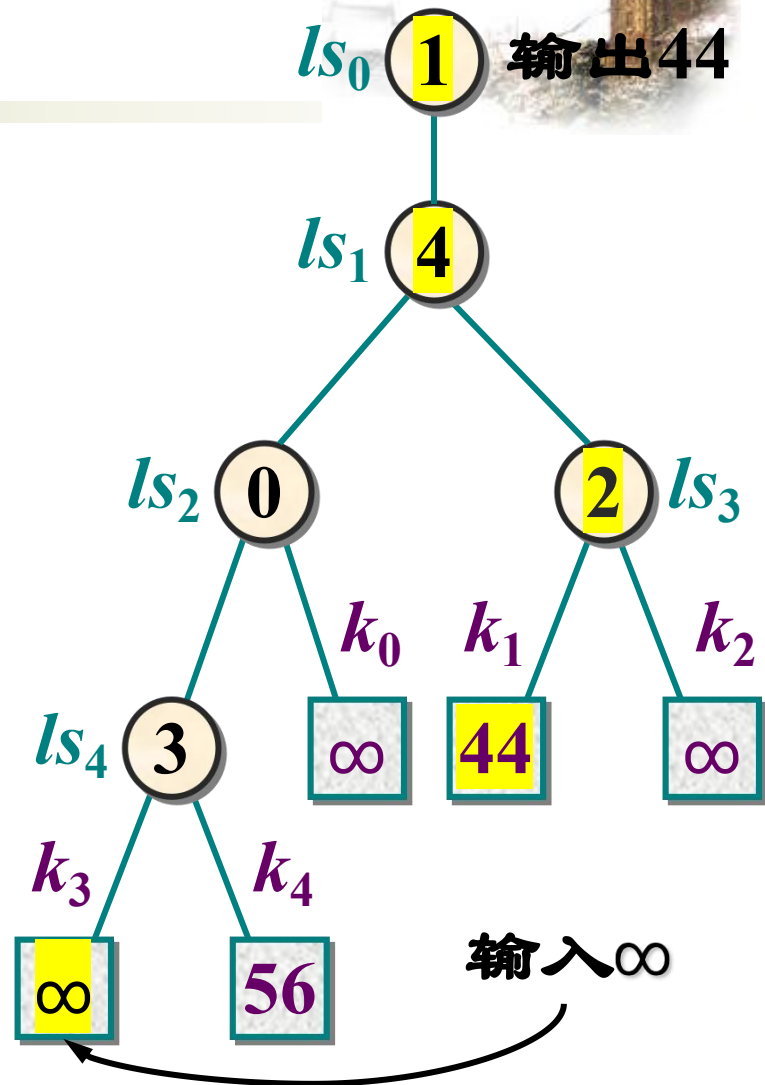
(11) 输出17后调整



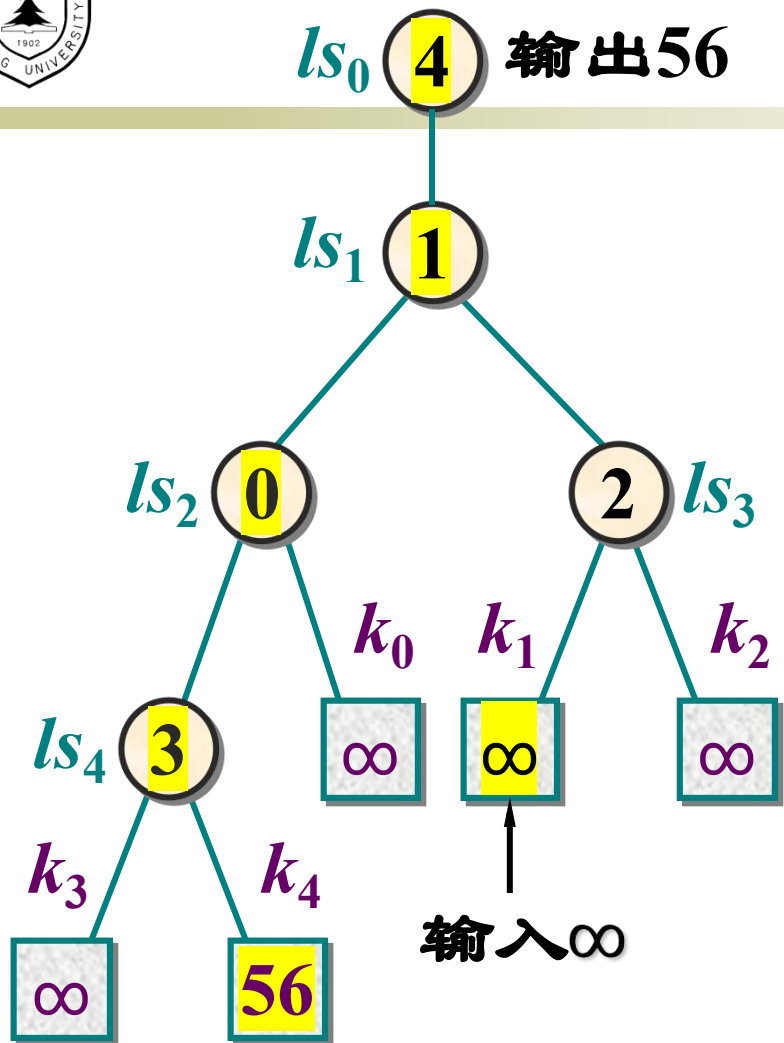
(12) 输出21后调整



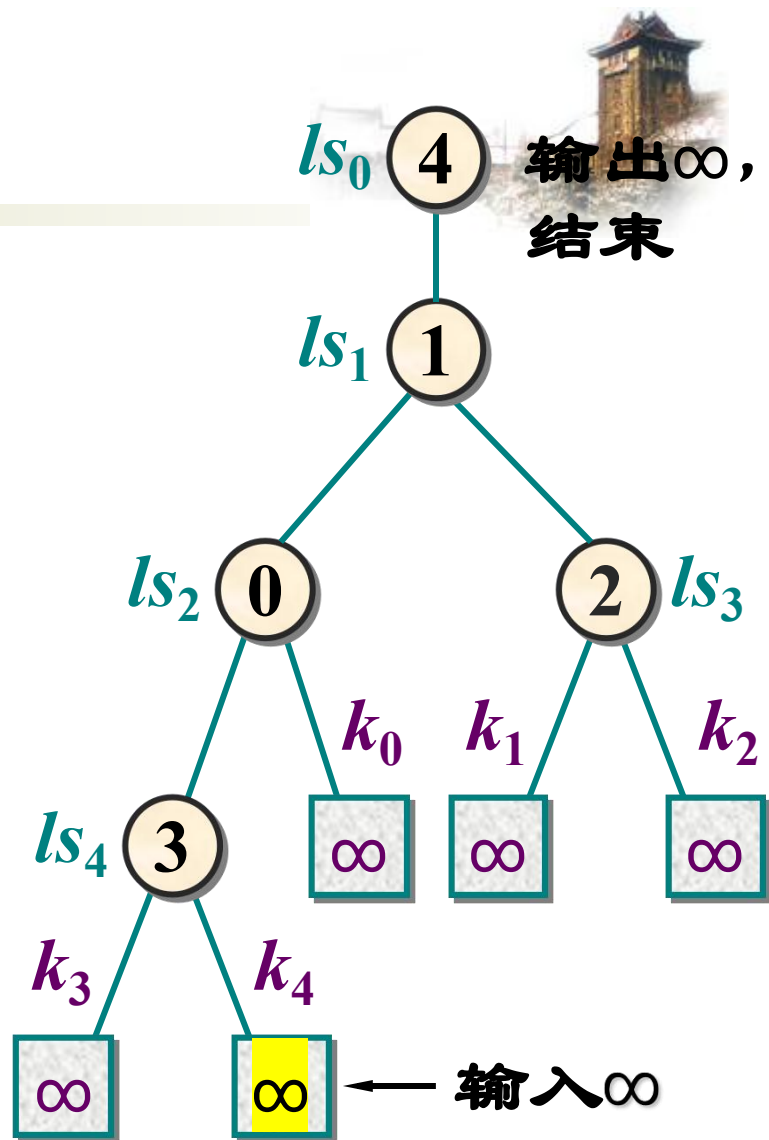
(13) 输出29后调整



(14) 输出32后调整



(15) 输出44后调整



(16) 输出56后调整



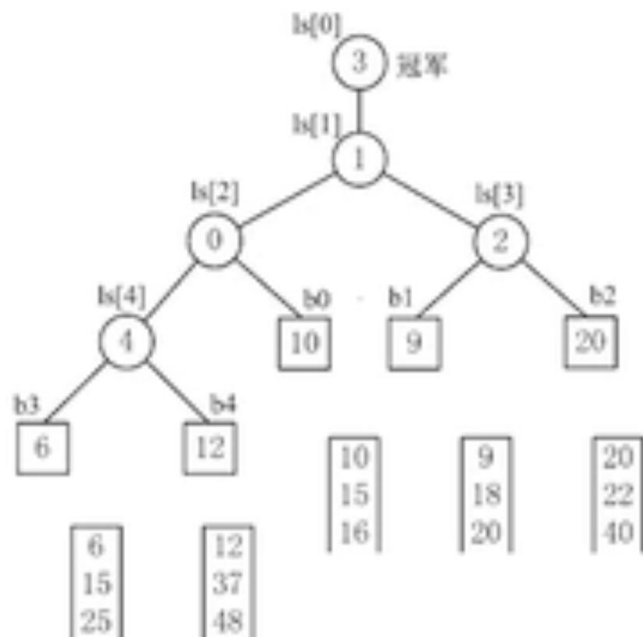
- 归并路数 k 不是越大越好。
- 归并路数 k 增大, 相应需增加输入缓冲区个数。
- 如果可供使用的内存空间不变, 势必要减少每个输入缓冲区的容量, 使内外存交换数据的次数增大。



败者树解决的问题：使用多路平衡归并可减少归并趟数，但是用老土方法从 k 个归并段选出一个最小/最大元素需要对比关键字 $k-1$ 次，构造败者树可以使关键字对比次数减少到 $\lceil \log_2 k \rceil$

败者树可视为一棵完全二叉树（多了一个头头）。 k 个叶结点分别对应 k 个归并段中当前参加比较的元素，非叶子结点用来记忆左右子树中的“失败者”，而让胜者往上继续进行比赛，一直到根结点。

如何构造和使用败者树？——看图记忆





最佳归并树

理论基础

每个初始归并段对应一个叶子结点，把归并段的块数作为叶子的权值

归并树的 WPL = 树中所有叶结点的带权路径长度之和

归并过程中的磁盘 I/O 次数 = 归并树的 WPL * 2

注意：k 叉归并的最佳归并树一定是严格 k 叉树，即树中只有度为 k、度为 0 的结点

如何构造

补充虚段

① 若 $(\text{初始归并段数量} - 1) \% (k - 1) = 0$ ，说明刚好可以构成严格 k 叉树，此时不需要添加虚段

② 若 $(\text{初始归并段数量} - 1) \% (k - 1) = u \neq 0$ ，则需要补充 $(k - 1) - u$ 个虚段

构造 k 叉哈夫曼树

每次选择 k 个根节点权值最小的树合并，并将 k 个根节点的权值之和作为新的根节点的权值



不考的章节



- 1.5.5
- 2.3.5 2.5.3 2.5.4
- 3.2.3 3.3.5 3.5
- 4.3.3 4.4.7 4.5.5
- 5.4.4 5.7.4 5.9.3
- 6.3 6.4 6.5.5
- 7.1.4 7.2.6 7.4 7.5
- 8.2.3 8.3.4 8.5.2
- 9.3.3 9.3.4 9.6 9.7 9.8.1
- 10.1 10.2 10.3 10.4.1 10.4.7 10.5 10.6